

TRƯỜNG ĐẠI HỌC SÀI GÒN  
KHOA CÔNG NGHỆ THÔNG TIN



---

Các công nghệ lập trình hiện đại

---

**Tìm hiểu LlamaIndex và xây dựng hệ  
thống hỏi đáp RAG về Quy chế đào tạo  
Trường Đại học Sài Gòn**

---

**GVHD:** Phạm Thi Vương

**SV:** Huỳnh Thanh Tiến 3123410374  
Phan Hữu Kha 3123560038  
Võ Thị Như Ý 3123410439  
Trần Hà Thanh Thùy 3123560087

Báo cáo tiến độ tuần 5 (09/10/2026)

Repo: [https://github.com/huynhthanhtien/LlamaIndex\\_RAG](https://github.com/huynhthanhtien/LlamaIndex_RAG)

Commit niêm phong: chưa niêm phong (bản nộp tiến độ tuần 5)

TP. HỒ CHÍ MINH, THÁNG 10/2026

# MỤC LỤC

|                                                           |           |
|-----------------------------------------------------------|-----------|
| <b>Lời mở đầu</b>                                         | <b>1</b>  |
| <b>1 Giới thiệu về LlamaIndex</b>                         | <b>5</b>  |
| 1.1 Bối cảnh và lịch sử . . . . .                         | 5         |
| 1.1.1 Bài toán: LLM không biết dữ liệu riêng . . . . .    | 5         |
| 1.1.2 LlamaIndex ra đời từ đâu . . . . .                  | 5         |
| 1.2 Kiến trúc và nguyên lý hoạt động . . . . .            | 6         |
| 1.2.1 Retrieval-Augmented Generation (RAG) . . . . .      | 6         |
| 1.2.2 Embedding và độ tương đồng vector . . . . .         | 7         |
| 1.2.3 Cắt đoạn văn bản (chunking) . . . . .               | 7         |
| 1.2.4 Chỉ mục vector và truy hồi top- $k$ . . . . .       | 8         |
| 1.2.5 Tổng hợp câu trả lời (Response Synthesis) . . . . . | 8         |
| 1.2.6 Xếp hạng lại (reranking) . . . . .                  | 9         |
| 1.2.7 Các thành phần chính và luồng xử lý . . . . .       | 9         |
| 1.2.8 Hệ sinh thái . . . . .                              | 11        |
| 1.3 So sánh và đánh giá . . . . .                         | 11        |
| 1.3.1 So với tự viết RAG . . . . .                        | 11        |
| 1.3.2 So với LangChain . . . . .                          | 11        |
| 1.3.3 Điểm mấu chốt quan sát được khi dùng thật . . . . . | 12        |
| 1.3.4 Khi nào KHÔNG nên dùng LlamaIndex . . . . .         | 14        |
| <b>2 Vị thế công nghệ và phạm vi ứng dụng</b>             | <b>15</b> |
| 2.1 Các trường hợp ứng dụng tiêu biểu . . . . .           | 15        |
| 2.2 Tình hình sử dụng . . . . .                           | 15        |
| 2.3 Xu hướng và giới hạn . . . . .                        | 15        |
| 2.4 Kết luận phạm vi . . . . .                            | 16        |
| <b>3 Thiết lập môi trường và kiến trúc dự án</b>          | <b>17</b> |
| 3.1 Công cụ cần thiết . . . . .                           | 17        |
| 3.1.1 Thư viện Python . . . . .                           | 17        |
| 3.1.2 Phụ thuộc hệ thống . . . . .                        | 18        |
| 3.1.3 LLM chạy qua Ollama . . . . .                       | 19        |
| 3.1.4 Phần cứng và bộ nhớ GPU . . . . .                   | 19        |
| 3.2 Cài đặt và Hello World . . . . .                      | 20        |
| 3.2.1 Các bước cài đặt . . . . .                          | 20        |
| 3.2.2 Hello World . . . . .                               | 20        |

|          |                                                          |           |
|----------|----------------------------------------------------------|-----------|
| 3.3      | Mình chứng kiến trúc . . . . .                           | 21        |
| 3.3.1    | Cấu trúc thư mục dự án . . . . .                         | 21        |
| 3.3.2    | Cấu hình tập trung và hai profile . . . . .              | 22        |
| 3.3.3    | Các script hỗ trợ . . . . .                              | 23        |
| 3.4      | Kiểm chứng môi trường . . . . .                          | 27        |
| <b>4</b> | <b>Bản chất công nghệ — Bắt buộc lỗi</b>                 | <b>28</b> |
| 4.1      | Nạp dữ liệu (Ingestion) . . . . .                        | 28        |
| 4.1.1    | Vì sao cần OCR . . . . .                                 | 28        |
| 4.1.2    | Cơ chế . . . . .                                         | 29        |
| 4.1.3    | Kết quả chạy thật . . . . .                              | 29        |
| 4.1.4    | Ý nghĩa trong đồ án . . . . .                            | 30        |
| 4.2      | Chia Node/chunk . . . . .                                | 30        |
| 4.2.1    | Cơ chế . . . . .                                         | 30        |
| 4.2.2    | Chunk size và overlap ở mức ứng dụng . . . . .           | 31        |
| 4.3      | Embedding và lập chỉ mục . . . . .                       | 31        |
| 4.3.1    | Mô hình embedding tiếng Việt được chọn . . . . .         | 31        |
| 4.3.2    | Cơ chế . . . . .                                         | 32        |
| 4.3.3    | Kết quả chạy thật . . . . .                              | 32        |
| 4.4      | Truy hồi (Retriever) . . . . .                           | 33        |
| 4.4.1    | Cơ chế . . . . .                                         | 33        |
| 4.4.2    | Kết quả chạy thật: top-5 kèm điểm . . . . .              | 33        |
| 4.5      | Tổng hợp câu trả lời (Response synthesis) . . . . .      | 35        |
| 4.5.1    | Cơ chế . . . . .                                         | 35        |
| 4.5.2    | Sửa lỗi nhầm thang điểm hệ 10 và hệ 4 . . . . .          | 36        |
| 4.5.3    | Hạn chế của cấu hình này . . . . .                       | 37        |
| 4.6      | Đánh giá thực nghiệm . . . . .                           | 37        |
| 4.6.1    | Độ đo . . . . .                                          | 37        |
| 4.6.2    | Kết quả . . . . .                                        | 38        |
| 4.6.3    | Câu hỏi tự kiểm . . . . .                                | 40        |
| <b>5</b> | <b>Custom node parser theo cấu trúc văn bản pháp quy</b> | <b>41</b> |
| 5.1      | Vì sao chọn . . . . .                                    | 41        |
| 5.2      | Cơ chế . . . . .                                         | 41        |
| 5.2.1    | Mã nguồn . . . . .                                       | 42        |
| 5.3      | Đã làm gì . . . . .                                      | 42        |
| 5.4      | Số liệu . . . . .                                        | 43        |
| 5.5      | Kiểm thử . . . . .                                       | 43        |

|           |                                                                 |           |
|-----------|-----------------------------------------------------------------|-----------|
| 5.6       | Hạn chế đã biết . . . . .                                       | 43        |
| 5.7       | Dùng ở đâu trong đồ án . . . . .                                | 43        |
| <b>6</b>  | <b>Persistent index và quản lý cấu hình</b>                     | <b>44</b> |
| 6.1       | Vì sao chọn . . . . .                                           | 44        |
| 6.2       | Cơ chế . . . . .                                                | 44        |
| 6.2.1     | Fingerprint: chống dùng lại chỉ mục cũ . . . . .                | 44        |
| 6.3       | Số liệu . . . . .                                               | 45        |
| 6.4       | Dùng ở đâu trong đồ án . . . . .                                | 45        |
| <b>7</b>  | <b>Trích dẫn nguồn và từ chối câu hỏi ngoài corpus</b>          | <b>46</b> |
| 7.1       | Vì sao chọn . . . . .                                           | 46        |
| 7.2       | Cơ chế . . . . .                                                | 46        |
| 7.3       | Đã làm gì . . . . .                                             | 46        |
| 7.4       | Số liệu . . . . .                                               | 46        |
| 7.5       | Dùng ở đâu trong đồ án . . . . .                                | 47        |
| <b>8</b>  | <b>Kỹ thuật nâng cao đã chọn: hybrid search và xếp hạng lại</b> | <b>48</b> |
| 8.1       | Vì sao chọn . . . . .                                           | 48        |
| 8.2       | Cơ chế . . . . .                                                | 48        |
| 8.3       | Số liệu . . . . .                                               | 51        |
| 8.4       | Dùng ở đâu trong đồ án . . . . .                                | 53        |
| <b>9</b>  | <b>Phân tích và thiết kế hệ thống</b>                           | <b>54</b> |
| 9.1       | Mô tả bài toán . . . . .                                        | 54        |
| 9.2       | User story . . . . .                                            | 54        |
| 9.3       | Thiết kế . . . . .                                              | 54        |
| 9.3.1     | Sơ đồ pipeline . . . . .                                        | 54        |
| 9.3.2     | Mô hình dữ liệu . . . . .                                       | 55        |
| 9.3.3     | Luồng truy vấn . . . . .                                        | 55        |
| <b>10</b> | <b>Triển khai, tích hợp và minh chứng kết quả</b>               | <b>56</b> |
| 10.1      | Tích hợp hệ thống . . . . .                                     | 56        |
| 10.2      | Kết quả đạt được . . . . .                                      | 56        |
| 10.2.1    | Truy hồi . . . . .                                              | 56        |
| 10.2.2    | Câu trả lời . . . . .                                           | 56        |
| 10.2.3    | So sánh trước – sau các tối ưu . . . . .                        | 63        |
| 10.2.4    | Hướng dẫn chạy lại . . . . .                                    | 63        |
| 10.3      | Bảng phủ kỹ thuật . . . . .                                     | 63        |

|                                                                            |           |
|----------------------------------------------------------------------------|-----------|
| <b>11 Đánh giá và nhìn lại</b>                                             | <b>65</b> |
| 11.1 So sánh với mục tiêu ban đầu . . . . .                                | 65        |
| 11.2 Hạn chế . . . . .                                                     | 65        |
| 11.3 Phân tích lỗi . . . . .                                               | 66        |
| 11.3.1 Lỗi 1: LLM nhầm thang điểm hệ 10 với hệ 4 . . . . .                 | 66        |
| 11.3.2 Lỗi 2: CUDA out of memory khi nạp embedding cùng reranker . . . . . | 67        |
| 11.4 AI Disclosure . . . . .                                               | 67        |
| <b>12 Hướng phát triển</b>                                                 | <b>68</b> |
| 12.1 Cải tiến sản phẩm . . . . .                                           | 68        |
| 12.2 Định hướng bản thân . . . . .                                         | 69        |
| 12.3 Kết luận . . . . .                                                    | 69        |
| <b>Tài liệu tham khảo</b>                                                  | <b>70</b> |
| <b>A Nhật ký công việc hàng tuần</b>                                       | <b>72</b> |
| A.1 Tuần 5 (05–09/10/2026) . . . . .                                       | 73        |
| A.1.1 Công việc đã làm . . . . .                                           | 73        |
| A.1.2 Kết quả và khó khăn . . . . .                                        | 73        |
| A.1.3 Việc chuyển sang tuần 6 . . . . .                                    | 74        |
| <b>B Bảng tự kiểm nhanh ba tầng</b>                                        | <b>75</b> |
| <b>C Thực nghiệm học tập</b>                                               | <b>76</b> |

# LỜI MỞ ĐẦU

## 1. Lý do chọn đề tài

Quy chế đào tạo là văn bản mà sinh viên tra cứu thường xuyên trong suốt quá trình học: thời gian tối đa để hoàn thành khóa học, cách tính điểm học phần, điều kiện xét tốt nghiệp, thủ tục nghỉ học tạm thời hay chuyển ngành. Tại Trường Đại học Sài Gòn (SGU), các quy định này nằm trong nhiều văn bản dài, phần lớn là PDF bản scan, và có văn bản sửa đổi chỉ áp dụng cho một số khóa. Tìm theo từ khóa chỉ khớp khi người hỏi dùng đúng thuật ngữ của văn bản (“cấm thi” không tìm được “không đủ điều kiện dự thi”); hỏi thẳng một mô hình ngôn ngữ lớn (LLM) thì câu trả lời trôi chảy nhưng không có căn cứ và có thể bịa.

**LlamaIndex** được chọn vì nó chuyên cho đúng bài toán này — hỏi đáp trên dữ liệu riêng — và có sẵn các khâu nạp dữ liệu, cắt Node, lập chỉ mục, truy hồi, tổng hợp câu trả lời [1]; hệ sinh thái của nó tích hợp sẵn Ollama (chạy LLM cục bộ, dữ liệu không ra dịch vụ bên ngoài) và mô hình embedding tiếng Việt trên HuggingFace.

Định hướng nghề nghiệp của từng thành viên: (*Chưa làm — bổ sung ở bản báo cáo cuối*).

## 2. Mục tiêu báo cáo

**Mục tiêu kiến thức (cam kết).** Sáu mục lỗi Tầng 1A:

1. Ingestion: Document và metadata.
2. Node/chunk, giải thích chunk size và overlap.
3. Embedding và vector index.
4. Retriever top- $k$  / similarity.
5. Response synthesis có truy vết nguồn.
6. Đánh giá thực nghiệm ở khâu truy hồi.

**Mục tiêu kỹ năng (kế hoạch).** Tầng 1B: custom node parser theo Điều, persistent index có fingerprint, trích dẫn nguồn, từ chối câu hỏi ngoài corpus, nhiều nguồn (6 văn

bản). Tầng 3: hybrid BM25 + vector, xếp hạng lại bằng cross-encoder, đánh giá có khoảng tin cậy và kiểm định thống kê, model server cho embedding/reranker.

### 3. Phạm vi nghiên cứu

- **Corpus:** 6 văn bản SGU đang áp dụng — Quy chế đào tạo trình độ đại học 2021, QĐ 3183/2025 sửa đổi Quy chế, QĐ 810 (học cùng lúc hai chương trình), QĐ 2375/2022 (chuyển chương trình, ngành), quy định khối lượng đào tạo, quy định số tín chỉ tối thiểu kỹ sư CNTT — tổng 54 Node.
- **Mô hình:** embedding AITeamVN/Vietnamese\_Embedding chạy local (GPU 4 GB ở fp16 hoặc CPU); LLM qwen3 : 8b qua Ollama trên máy GPU thuê hoặc Google Colab.
- **Không làm:** tư vấn pháp lý, agent, fine-tune LLM. Giao diện web chỉ ở mức chạy thử (Gradio).

**Thay đổi phạm vi so với kế hoạch ban đầu.** Kế hoạch ban đầu là một văn bản (Quy chế 2021, 17 trang scan) với LLM qwen3 : 4b chạy hoàn toàn trên máy cá nhân. Corpus được mở rộng lên 6 văn bản để trả lời được nội dung sửa đổi; LLM chuyển sang chạy từ xa vì GPU 4 GB không chứa nổi qwen3 : 8b cùng embedding. Lý do chi tiết ở Chương 11.

#### 4. Kế hoạch và tiến độ 12 tuần

| Tuần | Nội dung chính                | Việc cụ thể của nhóm                                              | Trạng thái                        |
|------|-------------------------------|-------------------------------------------------------------------|-----------------------------------|
| 1    | Lập nhóm, chọn công nghệ      | Đăng ký LlamaIndex, hồ sơ RAG                                     | Xong                              |
| 2    | Môi trường, Hello World, repo | venv, Ollama, hello_world.py, check_model_memory.py               | Xong                              |
| 3–4  | Bắt buộc lỗi, nháp Ch. 3–4    | OCR, Node theo Điều, index; bộ câu hỏi 01/02                      | Xong; thay bằng Markdown hai tầng |
| 5    | Chốt bài toán, thiết kế luồng | 6 văn bản, sơ đồ pipeline; Colab; chạy lại 56 câu                 | Xong                              |
| 6–7  | Chọn thêm, thực nghiệm        | Retriever, query engine, Recall@k (xong sớm); thí nghiệm chunking | Đang làm                          |
| 8–9  | Tích hợp, Tầng 3              | Trích nguồn, rerank, hybrid (xong sớm); từ chối ngoài corpus      | Một phần                          |
| 10   | Kiểm thử, đo trước–sau        | 26 kiểm thử + CI (xong sớm); chấm câu trả lời tự động             | Một phần                          |
| 11   | Hands-on lab, chấm chéo       | Viết lab                                                          | Chưa                              |
| 12   | Hoàn thiện, niêm phong        | Báo cáo, slide, video; nộp $\geq 24$ giờ trước bảo vệ             | Chưa                              |

Bảng 1: Kế hoạch 12 tuần và trạng thái tại tuần 5. Người thực hiện từng tuần ghi ở Phụ lục A.

#### 5. Cấu trúc báo cáo

Chương 1–2 giới thiệu LlamaIndex, kiến trúc, so sánh dựa trên trải nghiệm của nhóm và phạm vi nên/không nên dùng. Chương 3–8 trình bày môi trường, sáu mục lỗi Tầng 1A với mã nguồn và kết quả chạy thật, rồi các kỹ thuật chọn thêm (node parser, persistent index, trích dẫn/từ chối) và nâng cao (hybrid, rerank). Chương 9–10 là phân tích thiết kế, triển khai, minh chứng và Bảng phủ kỹ thuật. Chương 11–12 đánh giá lại so với mục tiêu, phân tích lỗi, AI Disclosure và hướng phát triển. Cuối báo cáo là tài liệu tham khảo và các phụ lục: nhật ký tuần, bảng tự kiểm ba tầng và thực nghiệm học tập.

Mọi số liệu đều lấy từ việc chạy thật các script trong repository; phần chưa làm được đánh dấu bằng khung “Chưa triển khai” hoặc ghi chú “Chưa làm” kèm thời điểm



dự kiến bổ sung.

# CHƯƠNG 1. GIỚI THIỆU VỀ LLAMAINDEX

## 1.1 Bối cảnh và lịch sử

### 1.1.1 Bài toán: LLM không biết dữ liệu riêng

Mô hình ngôn ngữ lớn (Large Language Model, LLM) là mạng nơ-ron được huấn luyện trên lượng văn bản rất lớn để dự đoán token tiếp theo. Nhờ đó LLM có khả năng hiểu câu hỏi, diễn đạt lại, tóm tắt và suy luận đơn giản. Tuy nhiên, khi dùng LLM để trả lời về một văn bản nội bộ như quy chế của SGU, có bốn hạn chế cần xử lý:

1. **Thiếu tri thức riêng.** Quy chế của một trường cụ thể, nhất là bản ban hành gần đây, hầu như không nằm trong dữ liệu huấn luyện, hoặc chỉ xuất hiện rất mờ nhạt.
2. **Tri thức bị đóng băng.** Kiến thức của mô hình dừng ở thời điểm huấn luyện; khi quy chế được sửa đổi, mô hình không tự cập nhật.
3. **Ảo giác (hallucination).** Mô hình có xu hướng sinh ra câu trả lời nghe hợp lý nhưng sai, ví dụ bịa số tín chỉ hoặc số Điều.
4. **Khó kiểm chứng.** Câu trả lời không kèm nguồn nên người dùng không biết đối chiếu ở đâu.

Có thể khắc phục bằng cách tinh chỉnh (fine-tuning) mô hình trên văn bản quy chế, nhưng cách này tốn kém, phải huấn luyện lại mỗi khi văn bản đổi, và vẫn không loại bỏ hoàn toàn ảo giác. RAG đi theo hướng khác: giữ nguyên mô hình, đưa tri thức cần thiết vào *ngữ cảnh* (prompt) tại thời điểm hỏi.

### 1.1.2 LlamaIndex ra đời từ đâu

LlamaIndex do Jerry Liu khởi xướng, ra mắt cuối năm 2022 dưới tên *GPT Index*, và đổi tên thành LlamaIndex trong năm 2023 [2]. Dự án hiện do công ty LlamaIndex Inc. phát triển: thư viện mã nguồn mở (gói `llama-index-core` và các gói tích hợp) đi kèm các dịch vụ thương mại như LlamaCloud và LlamaParse.

Điểm xuất phát của dự án cũng chính là pain-point nêu trên: LLM không biết dữ liệu riêng, cửa sổ ngữ cảnh có hạn nên không thể nhét cả kho tài liệu vào prompt, và khi thiếu căn cứ thì mô hình bịa. LlamaIndex giải quyết bằng một *lớp dữ liệu* đứng giữa tài

liệu và LLM: nạp tài liệu, cắt thành đơn vị nhỏ, lập chỉ mục, rồi khi có câu hỏi thì truy hồi đúng phần cần thiết đưa vào prompt.

## 1.2 Kiến trúc và nguyên lý hoạt động

**Gọi đúng loại.** LlamaIndex là một **thư viện/framework dữ liệu cho ứng dụng LLM**. Nó không phải là mô hình ngôn ngữ, và cũng không phải “RAG”: RAG là một *kiến trúc* (truy hồi rồi sinh), còn LlamaIndex là công cụ cài đặt kiến trúc đó (và cả các kiến trúc khác như agent).

### 1.2.1 Retrieval-Augmented Generation (RAG)

#### 1.2.1.1 Ý tưởng

RAG [3, 4] tách bài toán thành hai bước: (1) *truy hồi* — từ kho tài liệu, tìm ra một số ít đoạn liên quan nhất tới câu hỏi; (2) *sinh* — đưa câu hỏi cùng các đoạn đó cho LLM và yêu cầu trả lời dựa trên chúng. Nếu ký hiệu  $q$  là câu hỏi,  $\mathcal{D}$  là tập các đoạn văn bản trong kho, thì:

$$C = \text{Retrieve}(q, \mathcal{D}, k), \quad a = \text{LLM}(\text{prompt}(q, C)), \quad (1.1)$$

trong đó  $C \subset \mathcal{D}$  là  $k$  đoạn được chọn và  $a$  là câu trả lời. Vì  $C$  lấy trực tiếp từ quy chế nên câu trả lời có căn cứ, và hệ thống có thể trích dẫn nguồn (số Điều, số trang) cho người dùng kiểm tra.

#### 1.2.1.2 Hai pha của một hệ RAG

Một hệ RAG hoạt động theo hai pha có tần suất chạy khác nhau:

- **Pha chuẩn bị (offline), chạy khi tài liệu thay đổi:** nạp tài liệu  $\rightarrow$  cắt thành các đoạn  $\rightarrow$  biến mỗi đoạn thành vector embedding  $\rightarrow$  lưu vào chỉ mục.
- **Pha truy vấn (online), chạy mỗi câu hỏi:** embedding câu hỏi  $\rightarrow$  tìm  $k$  đoạn gần nhất  $\rightarrow$  ghép prompt  $\rightarrow$  LLM sinh câu trả lời.

Chất lượng của hệ thống bị giới hạn bởi *đoạn văn tìm được*: nếu bước truy hồi bỏ sót Điều đúng thì LLM dù tốt đến đâu cũng không thể trả lời đúng. Vì vậy phần lớn công sức của đồ án tập trung vào các bước chuẩn bị dữ liệu (nạp, cắt đoạn) và truy hồi.

## 1.2.2 Embedding và độ tương đồng vector

### 1.2.2.1 Vector embedding

Embedding là phép ánh xạ một đoạn văn bản thành một vector số thực  $\mathbf{v} \in \mathbb{R}^d$  sao cho các đoạn có nghĩa gần nhau nằm gần nhau trong không gian vector. Với mô hình dạng bi-encoder [5], câu hỏi và đoạn văn được mã hóa *độc lập* bởi cùng một bộ mã hóa; nhờ vậy vector của toàn bộ kho tài liệu có thể tính trước và lưu lại, khi có câu hỏi chỉ cần mã hóa riêng câu hỏi. Chất lượng của mô hình embedding là yếu tố ảnh hưởng mạnh nhất tới kết quả truy hồi, và được xếp hạng công khai trên các benchmark như MTEB [6].

### 1.2.2.2 Độ đo cosine

Độ tương đồng giữa vector câu hỏi  $\mathbf{q}$  và vector đoạn văn  $\mathbf{d}$  thường được đo bằng cosine:

$$\text{sim}(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \|\mathbf{d}\|} = \frac{\sum_{i=1}^d q_i d_i}{\sqrt{\sum_{i=1}^d q_i^2} \sqrt{\sum_{i=1}^d d_i^2}}. \quad (1.2)$$

Giá trị càng gần 1 thì hai đoạn càng gần nhau về ngữ nghĩa. Cách khác là so khớp theo từ khóa xác suất kiểu BM25 [7]: mạnh ở token hiếm và ký hiệu (“B+”, “150 tín chỉ”) nhưng không hiểu diễn đạt khác nghĩa; vì hai cách sai ở hai chỗ khác nhau, chúng thường được trộn thành *hybrid search*. Khác với tìm theo từ khóa, so khớp vector cho phép “cắm thi” tìm được đoạn viết “không đủ điều kiện dự thi” vì hai cụm có embedding gần nhau.

## 1.2.3 Cắt đoạn văn bản (chunking)

### 1.2.3.1 Vì sao phải cắt

Có hai lý do phải cắt tài liệu thành các đoạn nhỏ. Thứ nhất, cửa sổ ngữ cảnh của LLM có hạn, và càng nhét nhiều văn bản không liên quan thì câu trả lời càng dễ lệch — mô hình còn có xu hướng bỏ qua thông tin nằm giữa ngữ cảnh dài [8]. Thứ hai, một vector embedding cho cả tài liệu sẽ “pha loãng” mọi ý; mỗi đoạn nên xoay quanh một chủ đề để vector của nó đại diện đúng.

### 1.2.3.2 Các chiến lược cắt

| Chiến lược                       | Ưu điểm                               | Nhược điểm                                     |
|----------------------------------|---------------------------------------|------------------------------------------------|
| Cắt theo độ dài cố định          | Đơn giản, kích thước đều              | Có thể cắt ngang câu hoặc ngang điều khoản     |
| Cắt theo câu (sentence splitter) | Không gây câu, có chồng lấp (overlap) | Vẫn không biết ranh giới điều khoản            |
| Cắt theo cấu trúc văn bản        | Mỗi đoạn là một đơn vị nghĩa trọn vẹn | Phụ thuộc định dạng văn bản, cần quy tắc riêng |

Bảng 1.1: So sánh ba chiến lược cắt đoạn.

Ứng dụng cụ thể của các chiến lược này vào quy chế SGU nằm ở Mục 4.2 và Chương 5.

### 1.2.4 Chỉ mục vector và truy hồi top- $k$

Sau khi có vector cho mọi đoạn, hệ thống cần lưu chúng theo cách tìm kiếm nhanh. *Chỉ mục vector* (vector index) lưu các cặp (vector, đoạn văn, metadata). Khi có câu hỏi, bộ truy hồi (retriever) tính độ tương đồng giữa vector câu hỏi và các vector trong chỉ mục rồi trả về  $k$  đoạn có điểm cao nhất (top- $k$ ) [9]. Với kho nhỏ như một quy chế (vài chục đến vài trăm đoạn), việc so sánh vét cạn với mọi vector là đủ nhanh; các cấu trúc tìm kiếm xấp xỉ chỉ cần khi kho rất lớn.

### 1.2.5 Tổng hợp câu trả lời (Response Synthesis)

Sau truy hồi, các đoạn tìm được phải được đưa cho LLM để sinh câu trả lời. LlamaIndex gọi bước này là *response synthesis* và cung cấp nhiều chế độ (`response_mode`). Các chế độ thường gặp:

- `refine`: gọi LLM lần lượt từng đoạn, mỗi lần tinh chỉnh câu trả lời cũ bằng đoạn mới; chính xác nhưng tốn nhiều lượt gọi.
- `compact`: gộp nhiều đoạn vào cùng một prompt sao cho vừa cửa sổ ngữ cảnh rồi mới gọi LLM, giảm số lượt gọi so với `refine`.
- `tree_summarize`: tóm tắt theo cấu trúc cây, phù hợp câu hỏi tổng hợp trên nhiều đoạn.

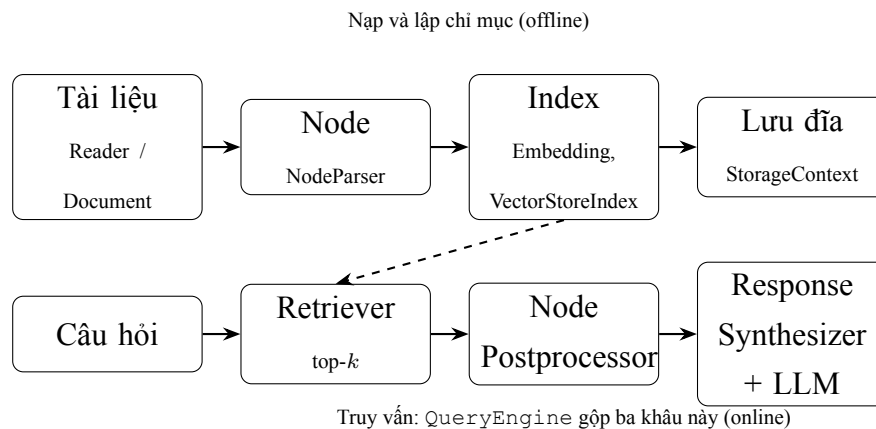
- `simple_summarize`: cắt bớt để nhét mọi đoạn vào một lần gọi.

### 1.2.6 Xếp hạng lại (reranking)

Mô hình embedding dạng bi-encoder mã hóa câu hỏi và đoạn văn *riêng rẽ*, nên nhanh nhưng chỉ so khớp ở mức “ý chính”. Mô hình *cross-encoder* nhận *cặp* (câu hỏi, đoạn văn) làm một đầu vào duy nhất và chấm một điểm liên quan; nhờ xem cả hai cùng lúc, nó thường đánh giá chính xác hơn, nhưng phải chạy lại cho từng cặp nên chậm hơn nhiều và không thể tính trước. Vì vậy cách dùng phổ biến là hai tầng: bi-encoder lấy nhanh top- $k$  ứng viên, rồi cross-encoder chấm lại và chỉ giữ top- $n$  ( $n < k$ ) đưa vào prompt.

### 1.2.7 Các thành phần chính và luồng xử lý

LlamaIndex [1] mô hình hóa chuỗi RAG bằng một số đối tượng có tên rõ ràng. Hình 1.1 vẽ lại luồng ba pha nạp — lập chỉ mục — truy vấn; bảng dưới chỉ ra từng đối tượng nằm ở đâu trong đồ án.



Hình 1.1: Luồng xử lý của LlamaIndex (vẽ lại theo tài liệu chính hãng).

| Khái niệm            | Vai trò                                                           | Trong đồ án                                                                           |
|----------------------|-------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Document             | Một nguồn dữ liệu thô kèm metadata                                | Bản đầu dùng ( <code>ingestion.py</code> ); nay chỉ còn ở <code>hello_world.py</code> |
| Node / TextNode      | Đơn vị được lập chỉ mục, sinh từ Document hoặc dựng trực tiếp     | <code>processed_loader.py</code> (mỗi Điều / mục sửa đổi một Node)                    |
| NodeParser           | Cắt Document thành Node ( <code>SentenceSplitter</code> , ...)    | <code>SentenceSplitter</code> cho Điều dài hơn 1 024 từ                               |
| Embedding            | Biến văn bản thành vector                                         | <code>HuggingFaceEmbedding</code> hoặc <code>RemoteEmbedding</code>                   |
| VectorStoreIndex     | Lưu Node kèm vector để truy hồi                                   | <code>index_builder.py</code>                                                         |
| StorageContext       | Ghi / nạp chỉ mục xuống đĩa                                       | <code>storage/&lt;profile&gt;/&lt;corpus&gt;</code>                                   |
| Retriever            | Lấy các Node liên quan tới câu hỏi                                | <code>retriever.py</code> (cosine $top-k$ , tùy chọn trộn BM25)                       |
| NodePostprocessor    | Lọc, xếp hạng lại Node sau truy hồi                               | <code>retriever.py</code> (ngưỡng, reranker)                                          |
| Response synthesizer | Ghép Node vào prompt và gọi LLM                                   | <code>query_engine.py</code> (chế độ compact)                                         |
| QueryEngine          | Gộp retriever, postprocessor và synthesizer                       | <code>query_engine.py</code> ( <code>RetrieverQueryEngine</code> )                    |
| Settings             | Cấu hình toàn cục ( <code>embed_model</code> , <code>llm</code> ) | Chỉ gán ở <code>index_builder</code> và <code>query_engine</code>                     |

Bảng 1.2: Các đối tượng chính của LlamaIndex và vị trí trong đồ án.

Điểm mạnh của thiết kế này là mỗi khâu độc lập: có thể đổi mô hình embedding, đổi cách cắt đoạn hoặc đổi chế độ tổng hợp mà không phải viết lại toàn bộ hệ thống.

### 1.2.8 Hệ sinh thái

Ngoài `llama-index-core`, LlamaIndex chia các tích hợp thành gói riêng: reader trên LlamaHub (PDF, web, CSDL...), vector store (Chroma, Qdrant, ...), LLM (Ollama, OpenAI, ...) và embedding (HuggingFace, ...). Đồ án dùng hai gói tích hợp: `llama-index-llms-ollama` và `llama-index-embeddings-huggingface`. Cách chia gói này có mặt trái: `BM25Retriever` không nằm trong core, nên đồ án tự viết (Mục 8.2).

## 1.3 So sánh và đánh giá

### 1.3.1 So với tự viết RAG

Đồ án có hai chỗ vừa dùng LlamaIndex vừa tự viết cùng chức năng, nên so sánh được bằng trải nghiệm thật:

- **Đánh giá truy hồi.** `eval/metrics.py` tự viết `Recall@k`, `MRR`, `nDCG`, `bootstrap` và `McNemar`; `RetrieverEvaluator` của LlamaIndex cho cùng `mrr` đến ba chữ số thập phân (Mục 4.6). Thư viện đủ cho `hit rate` và `MRR`, nhưng không có khoảng tin cậy và kiểm định cặp — phần đó vẫn phải tự viết.
- **BM25.** Tự viết khoảng 100 dòng (`src/bm25.py`) trên `BaseRetriever`; nhờ kế thừa interface của LlamaIndex, nó cắm thẳng vào `QueryFusionRetriever` mà không phải sửa phần còn lại.

Những gì LlamaIndex tiết kiệm được so với tự viết bằng `sentence-transformers` + `numpy`: lưu/nạp chỉ mục (`StorageContext`), chọn trường metadata nào vào embedding và prompt (các danh sách `excluded_..._metadata_keys`), trộn nhiều retriever, chuỗi postprocessor, response synthesizer có streaming và chế độ compact. Những gì nó *không* tiết kiệm được nằm ở bảng ma sát dưới đây.

### 1.3.2 So với LangChain

*(Chưa làm — nhóm chưa dựng lại pipeline bằng LangChain; dự kiến tuần 6–7 dựng cùng pipeline truy hồi để so số dòng code, cách xem Node/chunk, khả năng lưu chỉ mục và tài liệu)*

### **1.3.3 Điểm ma sát quan sát được khi dùng thật**

Phần này ghi lại những chỗ thư viện không làm đúng như kỳ vọng ban đầu. Đây là loại thông tin mà tài liệu chính thức không nêu, và là một trong những kết quả của việc *dùng thật* thư viện.

| Hiện tượng                                                              | Nguyên nhân                                                                                                                            | Cách xử lý                                                                                              |
|-------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Không nạp được embedding và reranker cùng lúc trên GPU 4 GB             | SentenceTransformerReRanker không có tham số dtype, luôn nạp fp32                                                                      | Nạp reranker trên CPU, ép <code>_model.half()</code> rồi mới chuyển lên GPU (dùng API nội bộ)           |
| Vector bị nhiễu bởi các trường metadata rỗng                            | Mặc định <code>metadata_template</code> là <code>{key}: {value}</code> cho mọi khóa, kể cả rỗng                                        | Lọc khóa rỗng và chỉ để một trường ( <code>nguồn_trích</code> ) được embedding                          |
| Không gán được nhãn tiếng Việt cho từng khóa metadata                   | <code>metadata_template</code> chỉ có hai chỗ điền <code>{key}</code> , <code>{value}</code> , không tùy biến theo khóa                | Gom nhãn vào một trường <code>nguồn_trích</code> tự sinh                                                |
| Bật hybrid là lỗi <code>ImportError: llama-index-llms-openai</code>     | <code>QueryFusionRetriever</code> làm <code>llm if llm else Settings.llm</code> , mà <code>Settings.llm</code> mặc định trỏ tới OpenAI | Truyền thẳng đối tượng Ollama vào retriever (với <code>num_queries=1</code> nó không bao giờ được gọi)  |
| <code>BM25Retriever</code> không có trong <code>llama-index-core</code> | Thư viện tách phần này ra package riêng ( <code>llama-index-retrievers-bm25</code> )                                                   | Tự viết <code>src/bm25.py</code> trên <code>BaseRetriever</code> (xem Mục 4.4)                          |
| Không lọc được theo toán tử “khác” trên metadata                        | <code>SimpleVectorStore</code> mặc định chỉ là JSON trong RAM, hỗ trợ lọc hạn chế                                                      | Chấp nhận ở quy mô hiện tại; ghi nhận việc chuyển sang Chroma/Qdrant khi corpus lớn                     |
| Đổi mô hình embedding là phải build lại toàn bộ chỉ mục                 | Định dạng lưu <code>default_vector_store.json</code> gắn với số chiều vector                                                           | Tách thư mục chỉ mục theo profile và thêm fingerprint theo model                                        |
| Cấu hình toàn cục gây tác dụng phụ                                      | <code>Settings</code> là trạng thái global ( <code>embed_model</code> , <code>llm</code> )                                             | Chỉ gán <code>Settings</code> ở đúng hai chỗ ( <code>index_builder</code> , <code>query_engine</code> ) |

Bảng 1.3: Các điểm ma sát với LlamaIndex gặp trong quá trình làm đồ án.

Một quan sát tổng quát: LlamaIndex tách bạch rất tốt ở mức *interface* (retriever,

node postprocessor, response synthesizer đều thay được độc lập), nhưng ở mức *mặc định* thì thiên về “chạy được ngay” hơn là “đúng cho tiếng Việt và cho văn bản pháp quy”. Vì vậy phần lớn công sức của đồ án nằm ở việc thay các mặc định đó, chứ không ở việc ghép các thành phần lại.

#### 1.3.4 Khi nào KHÔNG nên dùng LlamaIndex

1. **Câu hỏi cần tính toán hoặc tra bảng có cấu trúc.** Quy đổi điểm hệ 10 sang hệ 4 là phép tra bảng xác định. Đồ án ghi nhận LLM tự “quy đổi” 8,4 thành 3,4 và nhầm bảng hệ 10 với bảng hệ 4 (Mục 4.5.2) dù truy hỏi đúng. Loại câu này nên giải bằng một hàm Python hoặc truy vấn SQL trên bảng quy đổi, không bằng truy hỏi văn bản.
2. **Bài toán phân loại / dự đoán trên dữ liệu bảng.** Đó là việc của scikit-learn hay các thư viện học máy; LlamaIndex không có gì để đóng góp.
3. **Tài liệu ngắn, ít thay đổi, vừa cửa sổ ngữ cảnh.** Hai quy định một trang trong corpus nếu đứng riêng thì gọi thẳng LLM với toàn văn là đủ; dựng chỉ mục chỉ thêm độ phức tạp.

## CHƯƠNG 2. VỊ THẾ CÔNG NGHỆ VÀ PHẠM VI ỨNG DỤNG

### 2.1 Các trường hợp ứng dụng tiêu biểu

- **Hỏi đáp trên tài liệu nội bộ:** quy chế, sổ tay, tài liệu kỹ thuật — đúng dạng bài toán của đồ án.
- **Trợ lý tra cứu chính sách:** văn bản có sửa đổi, có hiệu lực theo thời gian, cần trích dẫn nguồn để người dùng tự kiểm tra.
- **Tìm kiếm tri thức doanh nghiệp:** gom nhiều nguồn (PDF, wiki, CSDL) vào một chỉ mục và hỏi bằng ngôn ngữ tự nhiên.

### 2.2 Tình hình sử dụng

*(Chưa làm — số liệu GitHub stars, lượt tải PyPI của llama-index-core và tin tuyển dụng sẽ thu thập kèm ngày truy cập ở tuần 6)*

### 2.3 Xu hướng và giới hạn

- **Xu hướng:** từ RAG một lượt sang *Workflows* (luồng nhiều bước điều khiển bằng sự kiện) và agent dùng công cụ; dịch vụ phân tích tài liệu (LlamaParse) cho PDF phức tạp.
- **Đối thủ:** LangChain/LangGraph (thiên về chuỗi và agent tổng quát), Haystack (thiên về pipeline tìm kiếm). LlamaIndex mạnh nhất ở tầng dữ liệu: nạp, cắt, lập chỉ mục, truy hỏi.
- **Giới hạn quan sát được:** mặc định thiên về tiếng Anh và OpenAI (`Settings.llm` trở tới OpenAI); nhiều tích hợp tách gói nên phải tự ghim phiên bản (Mục 1.3.3).

## 2.4 Kết luận phạm vi

| Nên dùng LlamaIndex                                                   | Không nên dùng                                     |
|-----------------------------------------------------------------------|----------------------------------------------------|
| Hỏi đáp trên kho văn bản riêng, cần trích nguồn                       | Tính toán, tra bảng xác định, truy vấn có cấu trúc |
| Kho tài liệu thay đổi, cần lập chỉ mục lại                            | Phân loại / dự đoán trên dữ liệu bảng              |
| Cần thử nhiều cách cắt, truy hồi, xếp hạng mà không viết lại hệ thống | Tài liệu ngắn vừa cửa sổ ngữ cảnh                  |

Bảng 2.1: Phạm vi nên và không nên dùng LlamaIndex, rút ra từ đề án.

## CHƯƠNG 3. THIẾT LẬP MÔI TRƯỜNG VÀ KIẾN TRÚC DỰ ÁN

### 3.1 Công cụ cần thiết

#### 3.1.1 Thư viện Python

File `requirements.txt` đã được **ghim đúng phiên bản** mà báo cáo dùng (trước đây chỉ ghi tên gói, người đọc cài sau sẽ nhận bản khác). Bảng dưới ghi phiên bản đo được trong `venv` tại thời điểm chạy lại toàn bộ số liệu.

| Gói                                | Phiên bản đo được  | Mục đích                                            |
|------------------------------------|--------------------|-----------------------------------------------------|
| llama-index-core                   | 0.14.24            | Framework RAG                                       |
| llama-index-embeddings-huggingface | 0.8.0              | Nạp mô hình embedding                               |
| llama-index-llms-ollama            | 0.11.0             | Kết nối LLM qua Ollama                              |
| sentence-transformers              | 6.0.1              | Chạy embedding/reranker                             |
| torch (cài kèm)                    | 2.14.0 (CUDA 13.0) | Chạy mô hình trên GPU                               |
| sentencepiece, protobuf            | 0.2.2, không đo    | Phụ thuộc của tokenizer                             |
| pytesseract                        | 0.3.13             | Gọi Tesseract từ Python                             |
| pdf2image                          | 1.17.0             | Render PDF thành ảnh                                |
| httpx                              | 0.28.1             | Gọi HTTP tới Ollama và model server                 |
| ollama                             | 0.6.2              | Client Python của Ollama (bắt lỗi trong app/app.py) |
| gradio                             | 6.27.0             | Giao diện web chạy thử (app/app.py)                 |
| python-dotenv                      | 1.2.3              | Đọc biến môi trường từ .env                         |

Bảng 3.1: Thư viện Python trong `requirements.txt`.

### 3.1.2 Phụ thuộc hệ thống

- **Tesseract OCR 5.5.3**, đã xác nhận có gói ngôn ngữ `vie`.
- **poppler-utils** (`pdftoppm 26.01.0`), cần cho `pdf2image`.

- **Ollama** để chạy LLM (script `pull_models.sh` kiểm tra và báo lỗi nếu chưa cài).
- Python 3.14.6 trong `venv`.

### 3.1.3 LLM chạy qua Ollama

Để dữ liệu không đi qua dịch vụ AI bên thứ ba, LLM được chạy qua **Ollama**, một trình phục vụ mô hình mở với API HTTP (mặc định tại `http://localhost:11434`); ở profile máy chủ, Ollama chạy trên một máy GPU thuê riêng và địa chỉ được đọc từ biến `OLLAMA_BASE_URL`. Đồ án dùng họ mô hình Qwen3: `qwen3:4b` cho profile chạy cục bộ và `qwen3:8b` cho profile máy chủ. LlamaIndex kết nối tới Ollama [10] qua gói `llama-index-llms-ollama`; Qwen3 là dòng mô hình mở của Alibaba, bản 4B và 8B đủ nhỏ để chạy trên máy cá nhân [11]. Thời gian chờ mặc định trong cấu hình là 180 giây do mô hình chạy trên phần cứng cá nhân có thể trả lời chậm. Qwen3 có chế độ “suy nghĩ” (sinh chuỗi lập luận trước câu trả lời); đồ án tắt chế độ này (`thinking=False`) để câu trả lời ngắn và nhanh.

### 3.1.4 Phần cứng và bộ nhớ GPU

Máy phát triển dùng card rời NVIDIA RTX A2000 Laptop (4 GB VRAM, khoảng 3,7 GB khả dụng cho PyTorch). Kiểm tra bằng cách nạp mô hình qua đúng hàm của dự án và in thiết bị của các tham số cho thấy cả mô hình embedding lẫn reranker đều nằm trên `cuda:0`. LLM không chạy trên máy này: ở profile `server`, Ollama chạy trên máy GPU thuê từ dịch vụ `ckey.vn`, được quản lý bằng `scripts/manager.py`.

Ở bản báo cáo trước, profile `server` **không nạp đồng thời** được embedding và reranker trên card 4 GB: tiến trình đã chiếm khoảng 3,5 GB và báo `CUDA out of memory` khi nạp mô hình thứ hai, nên phép đo phải đẩy embedding lên CPU. Bản hiện tại đã gộp thay đổi `fp16`: embedding còn khoảng 1,1 GB, cả hai mô hình cùng nằm trên GPU tổng khoảng 2,2 GB (xem `src/index_builder.py` và `make_local_reranker()` trong `src/retriever.py`). Thêm nữa, `resolve_device()` tự hạ cấp về CPU khi máy không có CUDA, nên cùng mã nguồn chạy được cả trên máy GPU, máy CPU và CI. Các số liệu ở Chương 11 được đo lại trên **CPU** vì môi trường chạy lại không có driver NVIDIA; đây là lý do thời gian truy hồi trong báo cáo lớn hơn con số đo trên GPU ở bản trước.

## 3.2 Cài đặt và Hello World

### 3.2.1 Các bước cài đặt

Trình tự đã dùng trong dự án:

1. Tạo và kích hoạt môi trường ảo Python (venv).
2. Cài các gói Python: `pip install -r requirements.txt`.
3. Cài Tesseract (kèm gói ngôn ngữ vie) và poppler-utils bằng trình quản lý gói của hệ điều hành.
4. Cài Ollama, chạy `ollama serve`, rồi tải mô hình bằng `scripts/pull_models.sh [local|server]`.

### 3.2.2 Hello World

Script `scripts/hello_world.py` kiểm tra nhanh rằng LlamaIndex cài đúng: tạo một Document, dựng SummaryIndex và đếm số Document trong docstore. Script này không cần LLM hay mô hình embedding nên chạy được ngay sau khi cài thư viện.

Đoạn mã 3.1: Tạo Document và SummaryIndex với LlamaIndex (`scripts/hello_world.py`).

---

```
1 from llama_index.core import Document, SummaryIndex
2
3 doc = Document(text = "Điều 15. Sinh viên nghỉ học quá 20% số tiết
    sẽ không đủ điều kiện dự thi.")
4 print("Document tạo thành công!")
5 print("Nội dung:", doc.text)
6 print("ID tự sinh:", doc.doc_id)
7
8 index = SummaryIndex.from_documents([doc])
9 print("\nIndex build thành công!")
10 print("Số Document trong Index:", len(index.docstore.docs))
```

---

Kết quả chạy thật:

Đoạn mã 3.2: Kết quả chạy `hello_world.py`.

---

```
1 Document tạo thành công!
```

```
2 Nội dung: Điều 15. Sinh viên nghỉ học quá 20% số tiết sẽ không đủ
   điều kiện dự thi.
3 ID tự sinh: 3dc12976-3b4e-4711-8798-c7d0237d98a9
4
5 Index build thành công!
6 Số Document trong Index: 1
```

---

Ý nghĩa: doc\_id được LlamaIndex tự sinh dạng UUID, và SummaryIndex lưu Document vào docstore. Câu ví dụ “Điều 15” trong script chỉ là văn bản minh họa tự viết, không phải nội dung trích từ quy chế.

### 3.3 Minh chứng kiến trúc

#### 3.3.1 Cấu trúc thư mục dự án

Đoạn mã 3.3: Cấu trúc thư mục dự án.

```
1 .
2 |-- app/          # Giao diện Gradio chạy thử (app/app.py)
3 |-- data/
4 |   |-- raw/      # PDF gốc (không commit)
5 |   |-- ocr/      # Tầng 1: Markdown thô sau OCR, nội sửa tay
6 |   |-- processed/ # Tầng 2: Markdown có heading, sinh lại từ tầng 1
7 |-- eval/
8 |   |-- evaluate.py      # Recall@k, MRR, nDCG, CI bootstrap,
   McNemar
9 |   |-- baselines.py     # baseline BM25 / không xếp hạng
10 |   |-- evaluate_answers.py # chấm câu trả lời bằng evaluator của
   LlamaIndex
11 |   |-- metrics.py      # đo đo + kiểm định thống kê
12 |   |-- cau_hoi_danh_gia.json # 13 câu có đáp án chuẩn + nhận phạm vi
13 |   |-- test_questions_01.json # 31 câu hỏi
14 |   |-- test_questions_02.json # 10 câu hỏi về QĐ sửa đổi 3183/2025
15 |   |-- results/        # kết quả JSON + Markdown
16 |-- notebooks/ # Chạy thử pipeline, chạy trên GPU thuê
17 |-- scripts/    # hello_world.py, test_ocr.py, check_model_memory.py,
18 |               # pull_models.sh, manager.py
19 |-- src/
20 |   |-- corpus.py      # Danh sách văn bản nạp vào index
21 |   |-- config.py      # RAGConfig + 2 profile + resolve_device()
22 |   |-- processed_loader.py # Markdown tầng 2 -> TextNode
23 |   |-- bm25.py        # BM25 thuần Python cho hybrid search
```

```
24 | |-- index_builder.py # Node -> VectorStoreIndex + fingerprint
25 | |-- retriever.py # vector / +BM25 / +rerank / ngưỡng tương đồng
26 | |-- query_engine.py # Prompt + LLM (temperature=0, seed) + nguồn
27 | |-- remote_models.py # Client gọi model server qua HTTP
28 | |-- model_server.py # FastAPI gửi embedding + reranker trên GPU
29 | |-- ingestion.py # BAN CU: OCR theo trang (không còn dùng)
30 | |-- node_parser.py # BAN CU: cắt theo regex (không còn dùng)
31 | `-- pipeline.py # Hoi dap qua dong lenh
32 | -- tests/ # 26 kiểm thử offline (unittest)
33 | -- storage/ # storage/<profile>/<corpus>/ (không commit)
34 | -- Makefile # make test / eval / baseline / eval-answers /
    report
35 `-- requirements.txt # đã ghim phiên bản
```

Mọi script chạy từ thư mục gốc của repository, ví dụ `RAG_PROFILE=server python src/pipeline.py`. Các module trong `src/` vừa là thư viện (được module sau import) vừa có khối `__main__` để chạy thử riêng từng giai đoạn.

### 3.3.2 Cấu hình tập trung và hai profile

Toàn bộ tham số RAG nằm trong `src/config.py` dưới dạng một dataclass `RAGConfig`. Có hai profile sẵn: `local` (máy cá nhân, mô hình nhỏ) và `server` (mô hình lớn hơn, có reranker, LLM chạy trên máy GPU thuê). Profile được chọn bằng biến môi trường `RAG_PROFILE` (đọc cả từ file `.env`), mặc định là `local`; giá trị không hợp lệ làm hàm `get_config()` ném `ValueError`.

Đoạn mã 3.4: Khai báo dataclass `RAGConfig` trong `src/config.py`.

```
1 @dataclass
2 class RAGConfig:
3     embed_model_name: str
4     embed_device: str # "cpu" or "cuda"
5     llm_model_name: str
6     ollama_base_url: str
7     request_timeout: float = 180.0
8     temperature: float = 0.0 # 0 = tắt định (sua loi tra loi khong
        on dinh)
9     seed: int = 42
10    reranker_model_name: str | None = None
11    reranker_top_n: int = 3
12    use_rerank: bool = False
13    similarity_top_k: int = 10
```

```
14 similarity_cutoff: float | None = None # ngưỡng tương đồng (None  
    = tat)  
15 use_hybrid: bool = False # BM25 + vector qua  
    QueryFusionRetriever  
16 bm25_top_k: int = 10  
17 fusion_mode: str = "reciprocal_rerank"  
18 response_mode: str = "compact"  
19 streaming: bool = True  
20 persist_dir: str = "" # trong = storage/<profile>/<corpus>
```

| Tham số            | Profile local                                                     | Profile server                                 |
|--------------------|-------------------------------------------------------------------|------------------------------------------------|
| Embedding          | bkai-foundation-<br>models/ vietnamese-<br>bi-encoder (768 chiều) | AITeamVN/Vietnamese_<br>Embedding (1024 chiều) |
| Thiết bị embedding | cuda                                                              | cuda                                           |
| LLM (Ollama)       | qwen3:4b,<br>localhost:11434                                      | qwen3:8b, địa chỉ lấy từ<br>OLLAMA_BASE_URL    |
| Reranker           | không dùng                                                        | AITeamVN/Vietnamese_<br>Reranker, giữ top 3    |
| Retriever          | top-10                                                            | top-10                                         |

Bảng 3.2: Hai profile cấu hình trong `src/config.py`.

Chỉ mục được lưu tách theo profile và tự build lại khi cấu hình đổi (Chương 6).

Nguồn dữ liệu thực tế được khai báo ở *một* chỗ: `src/corpus.py` (`CORPUS_DIR` và `dict SGU_HIEU_LUC`). Trường `corpus_sources` cũ cùng hai file PDF không tồn tại đã được gỡ bỏ, nên không còn tình trạng cấu hình trỏ vào file không có thật.

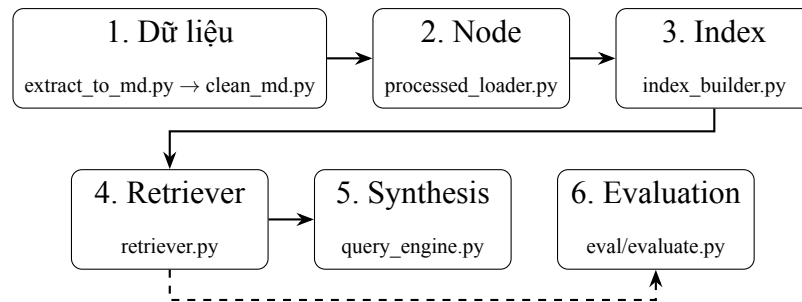
### 3.3.3 Các script hỗ trợ

- `scripts/test_ocr.py`: render toàn bộ một PDF và OCR ra file Mark-down trong `data/ocr/`, kèm số trang và tổng số ký tự. Đây là script tạo ra bản OCR `data/ocr/4. QuyCheDaoTaoDHSB 2021.md` mà `index_builder.py` dùng làm đầu vào.
- `scripts/check_model_memory.py`: đo RAM/VRAM khi nạp một mô

hình embedding hoặc reranker (nhận `repo_id`, `--type`, `--device`), giúp chọn mô hình vừa với phần cứng.

- `scripts/pull_models.sh`: tải LLM qua Ollama và tải embedding/reranker từ HuggingFace theo profile, đồng bộ với `config.py`; dừng với thông báo lỗi nếu thiếu `ollama`, `daemon` chưa chạy hoặc thiếu `huggingface_hub`.
- `scripts/manager.py`: công cụ dòng lệnh gom các thao tác với dịch vụ GPU thuê (xem, thuê, khởi động lại, xóa máy; xem số dư), gọi thử Ollama và chạy một server embedding.
- `scripts/setup_server_v2.sh`, `scripts/colab_server.py`: dựng phần chạy trên GPU (Ollama + model server embedding/reranker) trên Google Colab hoặc máy thuê, mở URL công khai qua Cloudflare quick tunnel và in sẵn các dòng `OLLAMA_BASE_URL`, `MODEL_SERVER_URL` để dán vào `.env` ở máy local. Thư viện của model server được ghim cùng phiên bản với `requirements.txt`, để vector sinh trên Colab khớp với chỉ mục đã build ở local. Nhờ vậy phần đánh giá câu trả lời — vốn cần LLM — chạy được mà không cần máy có GPU.
- `eval/run_batch_cau_hoi.py`: chạy hàng loạt danh sách câu hỏi trong `eval/results/cau-hoi01.md` qua query engine, ghi câu trả lời kèm nguồn ra `cau-hoi01-result.md` để chấm tay (Mục 10.2.2.4).
- `Makefile`: gom `make test` (26 kiểm thử offline), `make eval` (đo 4 cấu hình truy hồi), `make baseline`, `make eval-answers` và `make report`.
- `.github/workflows/ci.yml`: CI cài đúng phần tối thiểu (không torch, không model) và chạy toàn bộ kiểm thử offline trên mỗi lần push.

Chuỗi xử lý của đề án gồm sáu giai đoạn. Hình 3.1 thể hiện chuỗi này và file mã nguồn tương ứng; cả sáu giai đoạn đều đã có mã chạy được. Pha chuẩn bị (1–3) chạy một lần và lưu chỉ mục xuống đĩa; pha truy vấn (4–5) chạy mỗi câu hỏi; giai đoạn 6 dùng lại retriever của giai đoạn 4.



Hình 3.1: Sáu giai đoạn của pipeline RAG và file mã nguồn tương ứng; mũi tên nét đứt: bộ đánh giá gọi trực tiếp retriever.



| Giai đoạn             | Mức          | File                                                                                                                                            | Chạy được |
|-----------------------|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| 1. Dữ liệu            | Bắt buộc lỗi | scripts/<br>extract_<br>to_<br>md.<br>py,<br>scripts/<br>clean_<br>md.<br>py                                                                    | Có        |
| 2. Node               | Bắt buộc lỗi | src/ Có<br>processed_<br>loader.<br>py                                                                                                          |           |
| 3. Index              | Bắt buộc lỗi | src/ Có<br>index_<br>builder.<br>py                                                                                                             |           |
| 4. Retriever          | Bắt buộc lỗi | src/ Có<br>retriever.<br>py                                                                                                                     |           |
| 5. Response Synthesis | Bắt buộc lỗi | src/ Có<br>query_<br>engine.<br>py,<br>src/<br>pipeline.<br>py                                                                                  |           |
| 6. Evaluation         | Bắt buộc lỗi | eval/ Có (truy hỏi: tự động; câu trả lời: cần Ollama)<br>evaluate.<br>py,<br>eval/<br>baselines.<br>py,<br>eval/<br>evaluate_<br>answers.<br>py |           |
| Hybrid BM25           | Chọn thêm    | src/ Có (mặc định tắt)<br>bm25.<br>py,                                                                                                          |           |

### 3.4 Kiểm chứng môi trường

**Chỉ mục khớp với loader.** Chạy `src/index_builder.py` (profile server, CPU, ngày 09/10):

Đoạn mã 3.5: Kiểm chứng số Node trong chỉ mục.

```
1 [INFO] Load Index từ
    storage/server/01_quy_che_dao_tao/1_quy_che_dao_tao_dai_hoc
2 Index: storage/server/01_quy_che_dao_tao/1_quy_che_dao_tao_dai_hoc
    | số node trong Index: 54 | từ processed_loader: 54 | khớp: True
```

**Kết nối Ollama.** Một câu trả lời thật trong nhật ký Gradio (`eval/results/questions_log.jsonl`, ngày 08/10, qwen3:8b) cho câu “Sinh viên được rút học phần trong thời gian nào?”:

Đoạn mã 3.6: Câu trả lời mẫu qua Ollama.

```
1 Theo quy định tại Điều 7 –Quy chế đào tạo trình độ đại học SGU 2021
    (sửa đổi bởi
2 QĐ 3183/2025), việc rút bớt học phần trong khối lượng học tập đã
    đăng ký được thực
3 hiện trong 03 tuần đầu tiên của học kì.
```

Câu trả lời khớp khoản 4 mục sửa đổi Điều 7 của QĐ 3183/2025.

## CHƯƠNG 4. BẢN CHẤT CÔNG NGHỆ — BẮT BUỘC LỖI

Chương này đi qua sáu mục lỗi (Tầng 1A). Mỗi mục viết theo chuỗi: *cơ chế* → *mã nguồn* → *kết quả chạy* → *ý nghĩa trong đồ án*. Số liệu sinh lại được bằng `make test` và `make eval` trên đúng phiên bản thư viện đã ghim (Chương 3).

| TT | Yêu cầu lỗi                         | Cơ chế                                             | Minh chứng (file / log)                                                              | Vị trí trong đồ án             |
|----|-------------------------------------|----------------------------------------------------|--------------------------------------------------------------------------------------|--------------------------------|
| 1  | Ingestion, metadata                 | OCR bản scan, Mark-down hai tầng, front matter     | <code>scripts/extract_to_md.py</code> , <code>scripts/clean_md.py</code> ; 6 văn bản | Nguồn dữ liệu duy nhất         |
| 2  | Node/chunk, chunk size và overlap   | TextNode, cắt theo Điều, SentenceSplitter 1024/50  | <code>src/processed_loader.py</code> ; 54 Node                                       | Đơn vị truy hồi và trích dẫn   |
| 3  | Embedding + vector index            | HuggingFace embedding, VectorStoreIndex, persist   | <code>src/index_builder.py</code> ; 54/54 khớp                                       | Nền cho truy hồi               |
| 4  | Retriever top- $k$                  | Cosine, top- $k = 10$                              | <code>src/retriever.py</code> ; Bảng 4.2                                             | Chọn ngữ cảnh cho LLM          |
| 5  | Response synthesis + truy vết nguồn | RetrieverQueryEngine, chế độ compact, prompt riêng | <code>src/query_engine.py</code> ; dòng “Nguồn”                                      | Sinh câu trả lời               |
| 6  | Đánh giá thực nghiệm                | Recall@ $k$ , MRR, nDCG, khoảng tin cậy            | <code>eval/evaluate.py</code> ; R@1 81%, R@3 100%                                    | Chứng minh chất lượng truy hồi |

Bảng 4.1: Sáu mục lỗi và minh chứng tương ứng.

### 4.1 Nạp dữ liệu (Ingestion)

#### 4.1.1 Vì sao cần OCR

Bản PDF của Quy chế đào tạo SGU là bản scan: mỗi trang là một ảnh, không có lớp văn bản (text layer). Khi đọc bằng bộ đọc PDF thông thường, hệ thống gần như không lấy được chữ nào. Để có văn bản, cần OCR [12].

**Tesseract** [12] là công cụ OCR mã nguồn mở, có gói ngôn ngữ tiếng Việt (`vie`); trong Python được gọi qua thư viện `pytesseract`. Vì Tesseract nhận ảnh làm đầu vào, mỗi trang PDF trước hết được render thành ảnh bằng `pdf2image` (dựa trên `poppler`). Độ phân giải càng cao thì nhận dạng càng tốt nhưng chậm hơn; đồ án dùng 300 dpi.

OCR tiếng Việt luôn có sai sót, nhất là với dấu thanh và chữ in mờ (ví dụ nhầm

“NHÂN” thành “NHÂN”). Điều này ảnh hưởng đến các bước sau, đặc biệt khi quy tắc cắt đoạn dựa vào việc nhận đúng từ “Điều”. Mục 4.1.3 nêu chỉ số chất lượng OCR quan sát được.

#### 4.1.2 Cơ chế

LlamaIndex có sẵn `SimpleDirectoryReader` để đọc PDF, nhưng các văn bản quy chế của SGU là bản scan: trang PDF chỉ là ảnh, không có lớp văn bản nên bộ đọc thường trả về gần như rỗng. Vì vậy đồ án chuẩn bị dữ liệu ngoài LlamaIndex rồi mới đưa vào, theo hai tầng:

1. **Tầng 1 — `scripts/extract_to_md.py`**: nếu PDF có lớp văn bản thì trích trực tiếp; nếu không thì render từng trang ở 300 dpi bằng `pdf2image` rồi OCR bằng `pytesseract` với `lang="vie"` [12]. Script lưu kết quả vào `data/ocr/`, kèm chỉ số chất lượng là *tỷ lệ âm tiết tiếng Việt hợp lệ*; khi một văn bản có nhiều bản scan, chỉ số này dùng để chọn bản tốt hơn.
2. **Tầng 2 — `scripts/clean_md.py`**: làm sạch văn bản tầng 1 và dựng lại cấu trúc bằng heading Markdown: # cho tên văn bản, ## cho Chương, ### cho Điều, #### cho mục sửa đổi. Kết quả nằm ở `data/processed/`.

Tầng 1 là nơi *duy nhất* được sửa tay; tầng 2 luôn sinh lại được từ tầng 1, nên không có hai bản chân lý mâu thuẫn nhau. Cách tách này cũng là điều kiện để dùng lại toàn bộ hạ tầng LlamaIndex phía sau: mọi thứ từ node trở đi đều đi trên Markdown có cấu trúc.

**Metadata.** Mỗi file tầng 2 có khối front matter (`ten_van_ban`, `file_goc`, `url`, và các khóa tùy chọn `so_hieu`, `ngay_ban_hanh`, `trinh_trang`). `processed_loader` chuyển chúng thành metadata của Node — nền cho trích dẫn nguồn ở Mục 4.5.

#### 4.1.3 Kết quả chạy thật

Corpus hiện tại gồm 6 văn bản đang áp dụng. Chạy `python src/processed_loader.py` cho:

---

Đoạn mã 4.1: Số Node theo từng văn bản (sinh lại được bằng `make test`).

---

```
1 54 Node từ 6 văn bản trong
    data/processed/01_quy_che_dao_tao/1_quy_che_dao_tao_dai_hoc
2   5 2022-2375-QyD Chuyen Chuong Trinh Dao Tao DoiVoiSinhVenDHCQ.md
3   1
    22_quy-dinh-ve-khoi-luong-dao-tao-cua-cac-he-dao-tao-chinh-quy-dao-tao.m
4  25
    25_quy-che-dao-tao-trinh-do-dai-hoc-tai-truong-dai-hoc-sai-gon.md
5  13 8_QD810QuyDinhVeHocCungLucHaiChuongTrinhDoiVoiSVDHCQ.md
6   9 QD 3183.2025 - Sua doi Quy che Dao tao trinh do
    DH_LienQuanRutMonHoc.md
7   1 Quy dinh 150tc.md
```

Văn bản dài nhất (Quy chế 2021) cho 25 Node: 21 Điều của Quy chế cộng 4 Node thuộc phần “Quyết định ban hành”. Hai quy định một trang không chia Điều nên mỗi văn bản chỉ có một Node, đúng như thiết kế `MIN_TU_NOI_DUNG` (xem Mục 4.2).

Chất lượng OCR không đều: văn bản quan trọng nhất (QĐ 3183/2025) có tỷ lệ âm tiết hợp lệ thấp nhất (94,9%) và vẫn còn lỗi như “Sửa đổi”, “quy đổi” (xem trích đoạn ở Mục 4.5.2).

#### 4.1.4 Ý nghĩa trong đồ án

Toàn bộ hệ thống chỉ biết những gì đi qua bước này. Vì OCR sai được sửa tay ở tầng 1 và tầng 2 sinh lại tự động, chất lượng dữ liệu cải thiện dần mà không phải chạy lại OCR. Bài học từ bản đầu (`src/ingestion.py`, mỗi trang một Document) được trình bày ở Chương 5.

## 4.2 Chia Node/chunk

### 4.2.1 Cơ chế

Module `src/processed_loader.py` đọc Markdown tầng 2 và sinh `TextNode` — đối tượng đơn vị của `LlamaIndex` [1]:

1. Tách file theo heading bằng biểu thức chính quy `(?m)^(#{1,4})` (lookahead giữ lại dòng heading), rồi theo dõi cấp heading để biết Node thuộc văn bản nào, Chương nào.
2. Mỗi `###` (Điều) hoặc `####` (mục sửa đổi) là một Node; `nhân dieu` hoặc `sua_doi_dieu` được trích bằng `Điều (\d+)`.

3. Nội dung nằm ngay dưới `###` chỉ thành Node khi dài từ `MIN_TU_NOI_DUNG = 30` từ trở lên (dành cho văn bản không chia Điều).
4. Node dài hơn `MAX_TU = 1 024` từ được cắt tiếp bằng `SentenceSplitter` (`chunk_overlap=50`) và gắn thêm `phan_doan`.

#### 4.2.2 Chunk size và overlap ở mức ứng dụng

- **Đơn vị chính là Điều, không phải số token.** Một Điều của quy chế là một đơn vị nghĩa trọn vẹn (một thủ tục, một thang điểm). Cắt cố định theo token dễ chia đôi bảng điểm hoặc tách điều kiện khỏi hệ quả. Vì vậy `chunk_size` chỉ là *trần an toàn*: chỉ Điều dài hơn `MAX_TU = 1 024` từ mới bị cắt tiếp.
- **Vì sao 1 024.** Mô hình embedding của `profile server` nhận chuỗi dài; với 1 024 từ, hầu hết các Điều giữ nguyên một Node. Trên corpus hiện tại độ dài trung bình khoảng 1 500 ký tự/Node nên 10 Node vẫn vừa một lượt gọi LLM ở chế độ compact.
- **Overlap 50.** Chỉ có tác dụng khi một Điều bị cắt tiếp: phần chồng lấp giữ câu dẫn của khoản trước để đoạn sau không mất ngữ cảnh.
- **Đánh đổi.** Node to (cả Điều) giữ ngữ cảnh nhưng “pha loãng” vector và khiến LLM đọc sai chi tiết số trong Điều dài (lỗi “30 tuần” ở Chương 10); Node nhỏ (theo khoản) khớp chính xác hơn nhưng mất câu dẫn. Hướng dung hòa là small-to-big (Chương 12).

**Chưa triển khai.** Thí nghiệm so sánh các cách chunk (cố định 256/512/1024 token, theo câu, theo Điều, theo khoản) bằng `Recall@1/3/5` và `MRR` trên cùng bộ câu hỏi chưa chạy. Đây là việc của giai đoạn GD3.

### 4.3 Embedding và lập chỉ mục

#### 4.3.1 Mô hình embedding tiếng Việt được chọn

Chất lượng truy hồi phụ thuộc mạnh vào việc mô hình embedding có được huấn luyện cho tiếng Việt hay không. Theo cấu hình trong `src/config.py`, đề án dùng:

- Profile chạy cục bộ: `bkai-foundation-models/vietnamese-bi-encoder`, vector 768 chiều [13];
- Profile máy chủ: `AITeamVN/Vietnamese_Embedding`, vector 1024 chiều [14].

Cả hai đều được nạp qua `llama-index-embeddings-huggingface` (lớp `HuggingFaceEmbedding`, mặc định chuẩn hóa vector về độ dài 1 nên tích vô hướng chính là cosine). Vì hai mô hình cho vector khác số chiều, chỉ mục đã lập bằng mô hình này không dùng được với mô hình kia; đổi mô hình embedding đồng nghĩa phải lập lại chỉ mục. Phụ lục C ghi lại so sánh hai mô hình.

#### 4.3.2 Cơ chế

Module `src/index_builder.py` làm ba việc:

1. **Nạp Node** bằng `load_corpus_nodes()` của `processed_loader`.
2. **Đặt mô hình embedding** vào `Settings.embed_model` — `Settings` là cấu hình toàn cục của `LlamaIndex`, mọi thành phần cần embedding đều lấy từ đây. Hàm `resolve_device()` tự hạ cấp về CPU khi máy không có CUDA, nhờ đó cùng một mã nguồn chạy được trên máy GPU, trên CPU và trên CI.
3. **Dựng hoặc nạp lại** `VectorStoreIndex`. Chỉ mục được dựng thẳng từ danh sách `TextNode` đã cắt sẵn chứ không từ `Document`: nếu truyền `Document`, `LlamaIndex` sẽ tự cắt bằng `SentenceSplitter` mặc định và làm mất ranh giới Điều — đây chính là lý do phải gọi constructor bằng danh sách `Node`.

#### 4.3.3 Kết quả chạy thật

Với profile server (`AITeamVN/Vietnamese_Embedding`, 1024 chiều) trên corpus 6 văn bản, lần build đầu ghi ra:

Đoạn mã 4.2: Dòng log khi build lại chỉ mục (đo trên CPU).

```
1 Corpus, model embedding hoặc loader đã đổi so với index đã lưu –  
  build lại  
2 Generating embeddings: 100%|#####| 54/54 [01:17<00:00,  
  1.43s/it]
```

---

3 Index: storage/server/01\_quy\_che\_dao\_tao/1\_quy\_che\_dao\_tao\_dai\_hoc  
| 54 Node

---

Chỉ mục lưu theo storage/<profile>/<đường dẫn corpus>, nên hai profile có số chiều vector khác nhau không còn ghi đè lên nhau. Thời gian embedding 54 Node trên CPU khoảng 77 giây; trên GPU 4 GB (đo trước đây) nhanh hơn khoảng một bậc.

Cơ chế lưu và nạp lại chỉ mục, cùng fingerprint chống dùng nhầm chỉ mục cũ, được trình bày ở Chương 6.

## 4.4 Truy hồi (Retriever)

### 4.4.1 Cơ chế

Hàm `get_retriever()` trong `src/retriever.py` gọi `index.as_retriever()` với tham số `similarity_top_k = 10`: câu hỏi được embedding bằng đúng mô hình đã dùng khi lập chỉ mục, rồi so cosine với toàn bộ Node (`SimpleVectorStore` tìm vết cặn, đủ nhanh với vài chục đến vài trăm Node) và trả về 10 Node điểm cao nhất. Hai tầng tùy chọn — trộn BM25 và xếp hạng lại — thuộc phần nâng cao (Chương 8).

### 4.4.2 Kết quả chạy thật: top-5 kèm điểm

Chạy ngày 09/10, profile `server`, CPU, cấu hình mặc định (vector, không rerank):

| Hạng                                                      | Cosine | Node                                           |
|-----------------------------------------------------------|--------|------------------------------------------------|
| <i>“Một năm học có bao nhiêu học kỳ?”</i>                 |        |                                                |
| 1                                                         | 0,4377 | QĐ 3183/2025 – sửa đổi Điều 6                  |
| 2                                                         | 0,4336 | Quy chế 2021 – Điều 6                          |
| 3                                                         | 0,3713 | Quy chế 2021 – Điều 10                         |
| 4                                                         | 0,3666 | Quy chế 2021 – Điều 7                          |
| 5                                                         | 0,3202 | QĐ 3183/2025 – sửa đổi Điều 7                  |
| <i>“A thi được bao nhiêu điểm hệ 10”</i>                  |        |                                                |
| 1                                                         | 0,3843 | QĐ 3183/2025 – sửa đổi Điều 9                  |
| 2                                                         | 0,3605 | QĐ 3183/2025 – sửa đổi Điều 10                 |
| 3                                                         | 0,3373 | Quy chế 2021 – Điều 10                         |
| 4                                                         | 0,3334 | QĐ 3183/2025 – sửa đổi Điều 9 (phần đoạn khác) |
| 5                                                         | 0,2823 | Quy chế 2021 – Điều 9                          |
| <i>“Điều kiện để được học cùng lúc hai chương trình?”</i> |        |                                                |
| 1                                                         | 0,6192 | QĐ 810 – Điều 3                                |
| 2                                                         | 0,5895 | QĐ 810 – Điều 2                                |
| 3                                                         | 0,5641 | Quyết định ban hành QĐ 810 – Điều 2            |
| 4                                                         | 0,5603 | Quy chế 2021 – Điều 17                         |
| 5                                                         | 0,5502 | QĐ 810 – Điều 4                                |

Bảng 4.2: Top-5 Node của ba câu hỏi mẫu.

**Thảo luận.** Điểm cosine thấp (0,3–0,6) và các hạng sát nhau (hạng 1 và 2 của câu đầu chỉ chênh 0,004). Điều này có hai hệ quả: (1) không thể đặt một ngưỡng cố định để loại Node “không liên quan” mà không loại nhầm Node đúng — lý do `similarity_cutoff` mặc định tắt; (2) thứ tự giữa Điều gốc và mục sửa đổi của cùng một Điều gần như ngẫu nhiên, nên cả hai cần cùng nằm trong ngữ cảnh. Câu về điểm A cho thấy cả bảng hệ 10 (Điều 9) lẫn bảng hệ 4 (Điều 10) đều lọt vào top-5 — nguồn gốc của lỗi nhầm thang điểm ở Mục 4.5.2.

Trên bộ 31 câu hỏi về Quy chế 2021 (`profile server`, embedding trên CPU), cấu hình vector cho  $R@1 = 81\%$ ,  $R@3 = 100\%$ ,  $MRR = 0.892$ . Phân bố văn bản của Node top-1:

Đoạn mã 4.3: Văn bản của Node đứng đầu, bộ 01, cấu hình vector.

Quy chế đào tạo trình độ đại học SGU 2021 : 23 câu

- <sup>2</sup> QĐ 3183/2025 sửa đổi Quy chế đào tạo SGU : 4 câu  
<sup>3</sup> QĐ 810 về học cùng lúc hai chương trình : 3 câu  
<sup>4</sup> QĐ 2375/2022 về chuyển chương trình : 1 câu

Bốn câu còn lại cho thấy tác dụng phụ của việc mở rộng corpus: văn bản khác chủ đề nhưng dùng từ ngữ tương tự chen lên đầu. Trên bộ 02 (10 câu về nội dung sửa đổi) cả 10 câu đều lấy đúng QĐ 3183 ở top-1, nhưng câu hỏi về *khóa tuyển sinh áp dụng* thang điểm mới trượt khỏi top-3 vì đáp án nằm ở Điều 2 (điều khoản hiệu lực) trong khi truy hồi trả về các mục sửa đổi Điều 9, 10 — chúng khớp chữ “B+” nhiều hơn.

Kết quả đầy đủ của bốn cấu hình nằm ở Chương 8.

## 4.5 Tổng hợp câu trả lời (Response synthesis)

### 4.5.1 Cơ chế

Module `src/query_engine.py` ghép retriever, postprocessor và LLM thành `RetrieverQueryEngine`:

- **LLM**: `Ollama(model=..., thinking=cfg.thinking, context_window=8192)` [10, 11]. Mặc định tắt chế độ suy nghĩ của Qwen3 để câu trả lời ngắn; từ tuần 05–09/10 có thể bật bằng biến môi trường `RAG_THINKING=1` (trường `thinking` trong `RAGConfig`, được ghi vào `mo_ta()` để kết quả đánh giá biết đã đo ở chế độ nào). Bật chế độ này giúp tra bảng khoảng điểm cẩn thận hơn nhưng chậm khoảng bốn lần.
- **Tắt định**: `temperature=0` và `seed=42` truyền xuống Ollama. Đây là sửa lỗi đã đo được ở bản trước: cùng một câu hỏi, lần đầu hệ thống từ chối, lần sau trả lời “Không”, vì mô hình lấy mẫu ngẫu nhiên theo mặc định.
- **Prompt tiếng Việt riêng** thay prompt mặc định: nêu rõ ngữ cảnh là trích đoạn quy chế SGU, yêu cầu *chỉ* dựa vào trích đoạn, và bổ sung ba ràng buộc nhắm đúng các lỗi đã ghi trong nhật ký: (1) *trích nguyên văn câu quy định trước khi kết luận*, (2) *không tự quy đổi thang điểm hay đổi đơn vị, cần tra bảng thì chép đúng hàng của bảng*, (3) nói “không tìm thấy” khi trích đoạn không có thông tin — cho mô hình một lối ra hợp lệ thay vì phải đoán.
- **Trích dẫn nguồn**: `format_sources()` đọc metadata của các Node nguồn và

in ra “Nguồn: Điều X, Điều Y”. Nguồn lấy từ metadata chứ không để LLM tự ghi, nên luôn khớp với những gì đã đưa vào prompt.

- **Streaming:** câu trả lời in dần từng token qua `response.response_gen`.

#### 4.5.2 Sửa lỗi nhầm thang điểm hệ 10 và hệ 4

Nhật ký câu hỏi của giao diện Gradio (`eval/results/questions_log.jsonl`, mỗi câu hỏi một dòng JSON gồm câu hỏi, cấu hình, các Node nguồn và câu trả lời) ghi lại một lỗi lặp lại: với câu “A thì được bao nhiêu điểm hệ 10”, hệ thống trả lời “A được quy đổi thành 4,0 điểm hệ 10” ở cả hai lần hỏi (top-10 và top-3). Đáp án đúng là A ứng với khoảng 8,5–10 trên thang điểm 10 (khoản 3 Điều 9); con số 4,0 là giá trị quy đổi sang thang điểm 4 (khoản 2 Điều 10).

Truy hồi không sai: cả Điều 9 và Điều 10 (bản gốc lẫn bản sửa đổi của QĐ 3183) đều nằm trong ngữ cảnh. Nguyên nhân nằm ở dữ liệu: mục sửa đổi của QĐ 3183 chỉ chứa *bảng* (“A: từ 8,5 đến 10.0”, “A quy đổi thành 4,0”) mà không nhắc tên thang điểm, nên khi hai bảng đứng cạnh nhau trong ngữ cảnh, LLM không có căn cứ để biết bảng nào là “hệ 10” và bảng nào là “hệ 4”. Cách sửa là thêm vào QA\_PROMPT một đoạn chú giải miền (domain note) ngắn:

Đoạn mã 4.4: Chú giải thang điểm thêm vào QA\_PROMPT (`src/query_engine.py`).

- 1 Lưu ý về thang điểm: điểm thành phần và điểm học phần chấm theo thang điểm 10 ("**hệ 10**"),
- 2 rồi xếp loại thành điểm chữ (A, B+, B, ...) theo khoảng điểm ở khoản 3 Điều 9. Điểm chữ được
- 3 quy đổi sang thang điểm 4 ("**hệ 4**") chỉ để tính điểm trung bình, theo khoản 2 Điều 10.
- 4 Hỏi về hệ 10 hoặc khoảng điểm thì dùng bảng Điều 9; hỏi về hệ 4 hoặc quy đổi để tính điểm
- 5 trung bình thì dùng bảng Điều 10. Khi tra khoảng điểm, so sánh con số với cả cận dưới và
- 6 cận trên của từng hàng.

Đây là một đánh đổi có ý thức: prompt chứa kiến thức riêng của quy chế SGU, nên không dùng lại được cho văn bản khác. Nhưng thông tin “bảng nào thuộc thang nào” vốn có trong văn bản gốc (câu dẫn của khoản 3 Điều 9 và khoản 2 Điều 10) và chỉ bị mất khi cắt Node theo mục sửa đổi; cách sửa triệt để hơn là gắn câu dẫn đó vào metadata của mục sửa đổi (Mục 12.1). Hiệu quả của chú giải **chưa được đo lại**: chưa có phiên chạy

nào sau thay đổi này được ghi lại để xác nhận.

### 4.5.3 Hạn chế của cấu hình này

RetrieverQueryEngine chỉ gọi LLM *một lượt* trên ngữ cảnh đã truy hồi: nó không nhớ lịch sử hội thoại, nên câu hỏi nối tiếp kiểu “còn kỹ sư?” không đủ thông tin để truy hồi. Muốn sửa phải đổi sang chat engine có viết lại câu hỏi theo lịch sử (ví dụ CondensePlusContextChatEngine của LlamaIndex) — việc này được nêu ở Chương 12 như hướng phát triển, và đã có sẵn lịch sử chat trong giao diện Gradio để dùng.

## 4.6 Đánh giá thực nghiệm

### 4.6.1 Độ đo

Do chất lượng RAG phụ thuộc đầu tiên vào việc tìm đúng đoạn, đồ án đánh giá riêng bước truy hồi trước khi đánh giá câu trả lời. Có hai bộ câu hỏi kiểm thử, mỗi câu có hai trường `question` và `diều_dung` (số Điều chứa đáp án):

- `eval/test_questions_01.json`: 31 câu về Quy chế đào tạo 2021;
- `eval/test_questions_02.json`: 10 câu về các nội dung được *sửa đổi* bởi Quyết định 3183/2025 (rút học phần, số học kỳ, thang điểm chữ B+, C+, D+...).

Nhờ có nhãn, có thể dùng hai độ đo phổ biến. Gọi  $n_{i,j}$  là Node đứng thứ  $j$  trong danh sách trả về cho câu hỏi thứ  $i$ ,  $N$  là số câu hỏi:

- **Recall@ $k$**  (còn gọi là `hit rate@ $k$` ): tỷ lệ câu hỏi mà trong  $k$  Node đầu tiên có ít nhất một Node thuộc Điều đúng. Vì mỗi câu chỉ có một Điều đúng, hai cách gọi này trùng nhau:

$$\text{Recall}@k = \frac{1}{N} \sum_{i=1}^N \mathbb{1}[\exists j \leq k : \text{diều}(n_{i,j}) = \text{diều\_dung}_i]. \quad (4.1)$$

- **MRR** (Mean Reciprocal Rank): trung bình nghịch đảo thứ hạng của Node đúng đầu tiên (bằng 0 nếu không tìm thấy):

$$\text{MRR} = \frac{1}{N} \sum_{i=1}^N \frac{1}{\text{rank}_i}. \quad (4.2)$$

$\text{Recall}@k$  cho biết Điều đúng *có lọt vào* ngữ cảnh đưa cho LLM hay không; MRR cho biết nó *đúng cao* tới đâu. Đây là các độ đo chuẩn của bài toán truy hồi [15].

Cụ thể, bộ 01 (31 câu về Quy chế 2021) và bộ 02 (10 câu về nội dung sửa đổi bởi Quyết định 3183/2025). Vì corpus có nhiều văn bản trùng số Điều, mỗi bộ có luật riêng xác định Node nào là “đúng” (ví dụ bộ 01 tính cả Node gốc của Quy chế và Node sửa đổi tương ứng của QĐ 3183).

Bốn độ đo và hai loại kiểm định được cài trong `eval/metrics.py`:

- **$\text{Recall}@k$** : tỷ lệ câu có Node đúng trong  $k$  hạng đầu.
- **MRR**: trung bình nghịch đảo hạng của Node đúng đầu tiên.
- **$\text{nDCG}@k$** : như Recall nhưng tính cả vị trí, không chỉ “có hay không”.
- **Khoảng tin cây bootstrap** (2 000 lần lấy mẫu lại) cho mọi chỉ số — bắt buộc khi bộ câu hỏi chỉ có 10–31 câu.
- **Kiểm định McNemar chính xác**: so hai cấu hình trên *cùng* bộ câu hỏi, trả lời câu hỏi “chênh lệch này có thể chỉ do may mắn không”.

Ngoài ra `eval/evaluate.py --kiem-chung` chấm lại cùng bộ câu hỏi bằng `RetrieverEvaluator` của `LlamaIndex` (các metric `hit_rate`, `mrr`) để đối chiếu cách đo tự viết với cách đo của thư viện.

#### 4.6.2 Kết quả

**Môi trường đo.** Toàn bộ số liệu dưới đây được sinh lại bằng `make eval`, `make baseline` và `make test` trên máy chạy lại **không có GPU NVIDIA**, nên embedding và reranker chạy trên CPU (`resolve_device()` tự hạ cấp). Điều này không ảnh hưởng tới các chỉ số chất lượng (vector fp32 trên CPU so với fp16 trên GPU chỉ khác nhau ở mức làm tròn), nhưng làm thời gian mỗi câu lớn hơn nhiều so với con số đo trên GPU ở bản báo cáo trước. Mọi kết quả được ghi kèm cấu hình và thời điểm đo trong `eval/results/*.json`.

| Bộ          | Cấu hình          | R@1 (CI 95%)    | R@3  | R@10 | MRR   | nDCG@3 |
|-------------|-------------------|-----------------|------|------|-------|--------|
| 01 (31 câu) | vector (mặc định) | 81%<br>[68–94]  | 100% | 100% | 0,892 | 0,920  |
| 02 (10 câu) | vector (mặc định) | 90%<br>[70–100] | 90%  | 100% | 0,910 | 0,900  |

Bảng 4.3: Truy hồi ở cấu hình mặc định, profile server, 54 Node. Đo lại ngày 08/10 cho cùng chỉ số (eval/results/truy-hoi.md).

| Bộ          | Baseline           | R@1 (CI 95%)   | R@3 | R@10 | MRR   | ms/câu |
|-------------|--------------------|----------------|-----|------|-------|--------|
| 01 (31 câu) | BM25 (tìm từ khóa) | 58%<br>[42–74] | 84% | 97%  | 0,727 | 0,2    |
|             | không xếp hạng     | 0%             | 0%  | 0%   | 0,053 | 0,05   |
| 02 (10 câu) | BM25 (tìm từ khóa) | 40%<br>[10–70] | 60% | 90%  | 0,537 | 0,3    |
|             | không xếp hạng     | 0%             | 0%  | 0%   | 0,020 | 0,05   |

Bảng 4.4: Baseline (sinh lại bằng `make baseline`). “Không xếp hạng” trả về Node theo thứ tự docstore — tức móc sần khi không có bước xếp hạng nào.

**1. Mô hình embedding là yếu tố quyết định, và thiết kế “cắt theo Điều” là điều kiện để nó phát huy.** Trên bộ 01, vector đạt  $R@1 = 81\%$  và  $R@3 = 100\%$ , trong khi BM25 — đúng kiểu tìm kiếm từ khóa mà sinh viên đang dùng trên file PDF — chỉ đạt  $R@1 = 58\%$  và  $R@3 = 84\%$ . Khoảng cách 23 điểm phần trăm ở  $R@1$  là kết quả có ý nghĩa thực tiễn lớn nhất của đề án: nó định lượng được cái mà phần mở đầu chỉ nói bằng lời, rằng “cắm thi” không tìm được “không đủ điều kiện dự thi” bằng từ khóa.

**5. Kiểm chứng chéo bằng thư viện.** Chạy `eval/evaluate.py --kiem-chung`, chỉ số do `RetrieverEvaluator` của `LlamaIndex` tính khớp với chỉ số tự viết: bộ 01 `mrr=0,892` và bộ 02 `mrr=0,910` — trùng đúng đến ba chữ số thập phân, còn `hit_rate=1,000` ở cả hai bộ (tương ứng  $\text{Recall}@10 = 100\%$ ). Việc hai cách đo độc lập cho cùng kết quả là một kiểm tra tính đúng đắn của bộ đánh giá.

**6. Ca trượt còn lại và nguyên nhân.** Trên bộ 02, câu duy nhất nằm ngoài top-3 ở cấu hình vector là “Thang điểm chữ bổ sung (B+, C+, D+) và quy đổi điểm mới áp dụng cho khóa tuyển sinh nào?” — đáp án nằm ở Điều 2 (điều khoản hiệu lực) của QĐ 3183, nhưng truy hồi trả về các mục sửa đổi Điều 9 và 10 vì chúng chứa ký hiệu “B+” nhiều lần. Đây là dạng lỗi “khớp bề mặt thắng khớp ngữ nghĩa” đặc trưng của kho nhỏ, và là lý do nên bổ sung metadata hiệu lực (Mục 12.1).

#### 4.6.3 Câu hỏi tự kiểm

*“Cái gì trong đồ án này là LlamaIndex mà một dự án không dùng LlamaIndex sẽ không làm giống như vậy?”* Ba điều: (1) Node mang metadata và tự quyết định trường nào vào embedding, trường nào vào prompt (`excluded_embed_metadata_keys / excluded_llm_metadata_keys`); (2) truy hồi, hậu xử lý và tổng hợp là ba interface rời, nên BM25 tự viết và reranker cắm vào mà không đổi phần còn lại; (3) chỉ mục lưu/ nạp bằng `StorageContext` cùng `docstore`, nên Node truy hồi được luôn mang theo metadata để trích nguồn. Phần dữ liệu (OCR, Markdown hai tầng) thì *không* phải LlamaIndex — dự án nào cũng phải làm.

## CHƯƠNG 5. CUSTOM NODE PARSER THEO CẤU TRÚC VĂN BẢN PHÁP QUY

### 5.1 Vì sao chọn

Văn bản quy chế chia theo Chương — Điều — khoản, và văn bản sửa đổi (QĐ 3183) chia theo “mục sửa đổi Điều  $N$ ”. Câu trả lời và trích dẫn đều theo Điều, nên Node phải trùng ranh giới Điều. Bộ cắt mặc định của LlamaIndex (SentenceSplitter) không biết ranh giới này.

Giai đoạn 1 của bản báo cáo trước dùng `src/ingestion.py`: OCR từng trang và đóng gói *mỗi trang thành một Document*. Cách đó có hai nhược điểm đã đo được: (1) một Điều nằm vắt qua hai trang bị chia đôi; (2) phần “Quyết định ban hành” ở trang đầu sinh ra các Node trùng số Điều với phần Quy chế (Điều 1–3 xuất hiện hai lần), làm hỏng phép chấm truy hồi theo số Điều. Hai module `src/ingestion.py` và `src/node_parser.py` được giữ lại trong repository để đối chiếu nhưng **không còn nằm trong pipeline**; dữ liệu vào Node ngày nay đi qua tầng 2 nêu trên.

### 5.2 Cơ chế

Cơ chế tách theo heading được mô tả ở Mục 4.2. Phần dưới là hai quyết định về metadata — nơi custom parser khác hẳn parser mặc định.

**Loại metadata rộng.** Bản trước gán cho *mọi* Node bốn khóa `van_ban`, `chuong`, `dieu`, `sua_doi_dieu`, kể cả khi giá trị rỗng. Vì LlamaIndex ghép metadata vào văn bản trước khi embedding, mỗi vector đều chứa các dòng vô nghĩa kiểu `chuong:`, `dieu:` trong khi kho chỉ có vài chục Node. Bản mới lọc bỏ mọi khóa rỗng.

**Chỉ một trường được embedding.** Bản mới sinh thêm khóa `nguồn_trích` và chỉ trường này được đưa vào embedding; các khóa kỹ thuật còn lại bị loại bằng `excluded_embed_metadata_keys`. Văn bản đưa vào bộ mã hóa vì thế có dạng:

Đoạn mã 5.1: Văn bản đưa vào embedding của một Node (in bằng `metadata_mode="embed"`).

```
1 nguồn_trích: Điều 9 –Quy chế đào tạo trình độ đại học SGU 2021
```

```
2
```

---

3 <nguyên văn Điều 9>

---

Trước đây cùng Node này cho chuỗi `van_ban: ... \nchuong: \ndieu: 9 \nsua_doi_dieu:` — vừa nhiều vừa chứa khóa tiếng Anh không dấu. Chỉ số `nguồn_trich` cũng chính là nhãn `node_label()` dùng để in nguồn, nên phần hiển thị và phần embedding không còn lệch nhau. Đây là ví dụ cho thấy một quyết định nhỏ ở tầng Node ảnh hưởng trực tiếp tới chất lượng vector.

### 5.2.1 Mã nguồn

Đoạn mã 5.2: Gắn metadata đã lọc rỗng và chọn trường đưa vào embedding (`src/processed_loader.py`).

---

```
1 def _gan_metadata(node: TextNode, meta: dict[str, str | int]) ->
    TextNode:
2     """Gắn metadata đã lọc rỗng + `nguồn_trich`, và đặt đúng trường
        nào được embed / vào prompt."""
3     sach = {k: v for k, v in meta.items() if str(v).strip()}
4     sach["nguồn_trich"] = nguồn_trich(sach)
5     node.metadata = sach
6     node.excluded_embed_metadata_keys = [k for k in sach if k !=
        "nguồn_trich"]
7     node.excluded_llm_metadata_keys = [k for k in KHONG_LLM if k in
        sach]
8     return node
```

---

## 5.3 Đã làm gì

- Viết `src/processed_loader.py` thay cho `node_parser.py` (regex `DIEU_PATTERN` trên văn bản thô) — nay dựa vào heading do `clean_md.py` dựng.
- Tách riêng phần “Quyết định ban hành” (`phan="ban_hanh"`) để Điều 1–3 của quyết định không trùng nhãn với Điều 1–3 của quy chế.
- Gắn `LOADER_VERSION` để chỉ mục tự build lại khi logic cắt đổi (Chương 6).

## 5.4 Số liệu

54 Node từ 6 văn bản (Mục 4.1.3); chỉ mục và loader khớp 54/54 (Mục 3.4). Truy hồi đạt  $R@1$  81% trên bộ 01, cao hơn BM25 23 điểm phần trăm (Mục 4.6).

**Chưa triển khai.** Bảng so sánh cách chunk (B1) — xem Mục 4.2.

## 5.5 Kiểm thử

`tests/test_processed_loader.py` khóa lại đúng những hành vi trên: số Node và nhãn Điều, việc loại metadata rỗng, việc chỉ `nguồn_trích` được embedding, và giá trị của `nguồn_trích` cho ba trường hợp (Điều thường, mục sửa đổi, phần ban hành). `tests/test_corpus.py` kiểm tra trên corpus thật: mọi tiền tố trong danh sách văn bản khớp đúng một file, và mọi Node đều có `nguồn_trích`. Chạy `make test`: 26 kiểm thử, không cần GPU và không cần Ollama.

## 5.6 Hạn chế đã biết

- Cách cắt phụ thuộc `clean_md.py` dựng heading đúng. Nếu một văn bản bị OCR làm mất dòng “Điều *N*.” thì Điều đó bị gộp vào Điều trước hoặc vào Node Chương.
- “Token” vẫn đếm bằng số từ tách theo khoảng trắng, chỉ là xấp xỉ.
- Hai văn bản một trang cho mỗi văn bản đúng một Node dài, chưa cắt theo khoản.

## 5.7 Dừng ở đâu trong đồ án

Mọi bước sau đều đi trên Node này: chỉ mục, truy hồi, nhãn nguồn `node_label()`, và luật chấm đúng/sai theo (văn bản, Điều) của bộ đánh giá.

## CHƯƠNG 6. PERSISTENT INDEX VÀ QUẢN LÝ CẤU HÌNH

### 6.1 Vì sao chọn

Embedding 54 Node trên CPU mất khoảng 77 giây; dừng lại mỗi lần hỏi là không chấp nhận được. Ngược lại, dùng lại chỉ mục cũ sau khi đã đổi mô hình embedding hay cách cắt Node thì cho kết quả sai mà không báo lỗi. Cần lưu chỉ mục và biết khi nào nó hết hợp lệ.

### 6.2 Cơ chế

Hàm `build_or_load_index()` gọi `storage_context.persist()` sau khi dựng chỉ mục; ở các lần sau, nó tạo `StorageContext` từ thư mục đã lưu rồi gọi `load_index_from_storage()`. Thư mục lưu gồm docstore, vector store và index store dạng JSON.

#### 6.2.1 Fingerprint: chống dùng lại chỉ mục cũ

Bản trước chỉ lưu tên mô hình embedding và sự tồn tại của `docstore.json`, nên sửa cách cắt Node mà quên `--rebuild` sẽ âm thầm dùng chỉ mục cũ. Bản mới lưu `corpus.json` cạnh chỉ mục, gồm: model embedding, thư mục corpus, **phiên bản loader cùng các ngưỡng cắt**, và sha256 + tên hiển thị của từng file. Chỉ cần một trong các thứ đó đổi là chỉ mục tự build lại.

Đoạn mã 6.1: Fingerprint của chỉ mục (`src/index_builder.py`).

```
1  return {  
2      "embed_model": cfg.embed_model_name,  
3      "corpus_dir": cfg.corpus_dir,  
4      "loader": {"phien_ban": LOADER_VERSION, "max_tu": MAX_TU,  
5                 "min_tu_noi_dung": MIN_TU_NOI_DUNG},  
6      "files": files,  
7  }
```

Từ bản này, `persist_dir` được suy ra thành `storage/<profile>/<đường dẫn corpus>`, nên hai profile có số chiều vector khác nhau không còn ghi đè lên nhau (bản báo cáo trước ghi nhận đây là việc “nên làm”). Vì số chiều vector

gắn với mô hình, đổi mô hình embedding vẫn phải build lại chỉ mục — `corpus.json` cạnh chỉ mục sẽ tự phát hiện.

### 6.3 Số liệu

| Thao tác (profile <b>server</b> , CPU, 54 Node) | Thời gian                |
|-------------------------------------------------|--------------------------|
| Dựng mới (embedding toàn bộ Node)               | khoảng 77 giây           |
| Nạp lại từ đĩa (kể cả nạp mô hình embedding)    | 1,5 giây (đo ngày 09/10) |

Bảng 6.1: Thời gian dựng mới và nạp lại chỉ mục.

### 6.4 Dừng ở đâu trong đồ án

Mọi điểm vào (`pipeline.py`, `app/app.py`, `eval/*.py`, `notebook`) đều gọi `build_or_load_index()`, nên không có đường nào dừng nhằm chỉ mục cũ.

## CHƯƠNG 7. TRÍCH DẪN NGUỒN VÀ TỪ CHỐI CÂU HỎI NGOÀI CORPUS

### 7.1 Vì sao chọn

Câu trả lời về quy chế chỉ có giá trị khi người đọc kiểm tra lại được. Và với câu hỏi mà quy chế không nói tới, “không tìm thấy” là câu trả lời đúng — bịa ra một con số là lỗi nặng nhất.

### 7.2 Cơ chế

**Trích dẫn.** `format_sources()` đọc metadata của các Node nguồn qua `node_label()` và in “Nguồn: <văn bản> – Điều *N*”. Nguồn lấy từ metadata chứ không để LLM tự ghi, nên luôn khớp với những gì đã vào prompt. Prompt còn bắt LLM trích nguyên văn câu quy định kèm tên văn bản và số Điều trước khi kết luận.

**Từ chối.** Hai lớp: (1) prompt cho LLM một lỗi ra hợp lệ — “nếu trích đoạn không có thông tin, hãy nói không tìm thấy”; (2) `SimilarityPostprocessor` loại Node dưới ngưỡng (`similarity_cutoff`), đã cài nhưng mặc định tắt vì điểm cosine thấp và sát nhau (Mục 4.4).

### 7.3 Đã làm gì

- Nhân nguồn thống nhất với văn bản embedding qua trường `nguồn_trich` (Chương 5).
- Bộ `eval/cau_hoi_danh_gia.json`: 13 câu, mỗi câu có `phạm_vi` (10 “trong”, 3 “ngoài”) để tách từ chối đúng khỏi từ chối sai.

### 7.4 Số liệu

Trên 56 câu hỏi tự đặt (Chương 10), tỷ lệ trả lời “không tìm thấy” là 52% ở phiên đầu và 41% ở phiên chạy lại ngày 08/10. Chấm tay cho thấy cả hai loại: từ chối đúng (“điểm rèn luyện là gì”, “bao nhiêu tiền một tín chỉ”) và từ chối sai (“còn kỹ sư”, “rớt một môn có được xuất sắc không”).

**Chưa triển khai.** Tỷ lệ từ chối đúng trên một bộ câu ngoài corpus đủ lớn (ít nhất 10 câu) chưa đo; bộ hiện có mới 3 câu “ngoài”. Trích dẫn chưa tới mức khoản/trang.

## 7.5 Dừng ở đâu trong đồ án

Mọi câu trả lời ở CLI, giao diện Gradio và nhật ký đánh giá đều kèm dòng “Nguồn”.

## CHƯƠNG 8. KỸ THUẬT NÂNG CAO ĐÃ CHỌN: HYBRID SEARCH VÀ XẾP HẠNG LẠI

### 8.1 Vì sao chọn

Câu hỏi về quy chế chứa ký hiệu và con số mà embedding nắm kém (“B+”, “Điều 10”, “150 tín chỉ”), và bi-encoder chỉ so khớp ở mức ý chính. Hai kỹ thuật thường được khuyến nghị cho đúng hai điểm yếu này là trộn BM25 (hybrid search) và xếp hạng lại bằng cross-encoder. Đồ án cài cả hai và *đo* xem chúng có giúp trên corpus của mình không.

### 8.2 Cơ chế

Module `src/retriever.py` tổ chức truy hồi thành ba tầng độc lập, đúng theo cách LlamaIndex tách *retriever* khỏi *node postprocessor*:

1. **Vector top- $k$  (mặc định):** `index.as_retriever(similarity_top_k=10)`. Câu hỏi được embedding bằng đúng mô hình đã dùng khi lập chỉ mục, rồi so cosine với toàn bộ Node (SimpleVectorStore tìm vết cặn, phù hợp với kho vài chục đến vài trăm Node).
2. **Hybrid BM25 + vector (tùy chọn, `use_hybrid`):** câu hỏi về quy chế chứa nhiều ký hiệu và con số mà embedding nắm kém (“B+”, “Điều 10”, “150 tín chỉ”, “QĐ 3183”). Đồ án cài BM25 Okapi [7] thuần Python trong `src/bm25.py` — gói BM25Retriever chính thức nằm ở một package riêng chứ không có trong llama-index-core, nên tự viết vừa nhẹ vừa cho phép kiểm soát cách tách từ. Hai nguồn được trộn bằng QueryFusionRetriever ở chế độ `reciprocal_rerank` (RRF), với `num_queries=1` để không gọi LLM sinh truy vấn phụ và giữ phép đo tắt định.
3. **Postprocessor:** ngưỡng tương đồng SimilarityPostprocessor (tùy chọn) rồi tới xếp hạng lại bằng cross-encoder [16] (SentenceTransformerRerank, giữ top-3).

**Tách từ cho BM25.** `tach_tu()` bỏ dấu tiếng Việt trước khi chấm điểm (Điều → dieu). Nhờ vậy câu hỏi gõ thiếu dấu hoặc lỗi OCR vẫn khớp văn bản có dấu — đúng

loại lỗi đã gặp trong nhật ký hỏi đáp. Đánh đổi là mất phân biệt các cặp chỉ khác dấu, hiếm gặp trong văn bản quy chế. `tests/test_bm25.py` khóa lại hành vi này bằng ba ca: khớp theo ký hiệu “B+”, khớp khi câu hỏi không dấu, và trả về rỗng khi không có từ nào khớp (tránh đưa Node nhiều vào prompt).

**Một chi tiết dễ sai.** Điểm sau khi trộn RRF không cùng thang với cosine, nên nếu đặt `similarity_cutoff` khi đang bật hybrid thì mọi Node đều bị loại. Hàm `get_postprocessors()` vì thế chủ động bỏ qua ngưỡng khi `use_hybrid=True` và ghi log cảnh báo, thay vì trả về danh sách rỗng một cách im lặng.

**Chạy trên GPU 4GB và trên máy từ xa.** Reranker nạp ở fp16 (Mục 3.1.4). Khi đặt `MODEL_SERVER_URL`, embedding và reranker được gọi qua HTTP tới `src/model_server.py` (`RemoteEmbedding`, `RemoteRerank` trong `src/remote_models.py`); client kiểm tra server đang chạy đúng mô hình của profile để vector không lệch với chỉ mục.



### 8.3 Số liệu

| Bộ          | Cấu hình                    | R@1<br>(CI<br>95%)  | R@3  | MRR   | nDCG@1ms/câu |       |
|-------------|-----------------------------|---------------------|------|-------|--------------|-------|
| 01 (31 câu) | vector (mặc định)           | 81%<br>[68–<br>94]  | 100% | 0,892 | 0,806        | 165   |
|             | + rerank                    | 81%<br>[65–<br>94]  | 100% | 0,892 | 0,806        | 8 487 |
|             | hybrid (RRF, trọng số bằng) | 77%<br>[61–<br>90]  | 94%  | 0,866 | 0,774        | 155   |
|             | hybrid + rerank             | 81%<br>[65–<br>94]  | 100% | 0,892 | 0,806        | 8 193 |
|             | hybrid ưu tiên vector 3:1   | 84%<br>[71–<br>97]  | 100% | 0,914 | 0,839        | 148   |
| 02 (10 câu) | vector (mặc định)           | 90%<br>[70–<br>100] | 90%  | 0,910 | 0,900        | 149   |
|             | + rerank                    | 60%<br>[30–<br>90]  | 100% | 0,783 | 0,600        | 7 854 |
|             | hybrid (RRF, trọng số bằng) | 60%<br>[30–<br>90]  | 90%  | 0,717 | 0,600        | 166   |
|             | hybrid + rerank             | 60%<br>[30–<br>90]  | 90%  | 0,750 | 0,600        | 8 811 |
|             | hybrid ưu tiên vector 3:1   | 80%<br>[50–<br>100] | 90%  | 0,850 | 0,800        | 137   |

Bảng 8.1: Truy hồi trên corpus 6 văn bản / 54 Node, profile server (AITeamVN/Vietnamese\_Embedding), đo trên CPU. Sinh lại bằng make eval.

**2. Xếp hạng lại (rerank) không cải thiện trên kho nhỏ, và có hại cho bộ 02.** Trên bộ 01, rerank giữ nguyên cả ba chỉ số (81% / 100% / 0,892) nhưng đổi thứ tự top-1 ở bốn câu (số câu lấy đúng Quy chế 2021 tăng từ 23 lên 25). Trên bộ 02, rerank làm  $R@1$  tụt từ 90% xuống 60% và MRR từ 0,910 xuống 0,783, đổi lại  $R@3$  tăng từ 90% lên 100%. Nguyên nhân hợp lý: các mục sửa đổi của QĐ 3183 rất ngắn và dùng chung từ vựng (“thang điểm”, “điểm chữ”), nên cross-encoder đẩy mục sửa đổi “na ná” lên trên mục đúng; đổi lại nó kéo được Node đúng vào top-3. Về chi phí, trên CPU rerank làm mỗi câu từ 0,15 s lên 8,5 s — chậm hơn khoảng 50 lần — mà không đem lại chỉ số nào tốt hơn. Kết luận thực nghiệm: **giữ `use_rerank=False`** trên quy mô 54 Node, và chỉ đo lại khi corpus lớn hơn nhiều (hàng trăm đến hàng nghìn Node).

**3. Hybrid search: bản thân BM25 ở đây yếu, và cách trộn còn quan trọng hơn việc có trộn hay không.** Cấu hình hybrid mặc định (RRF, hai nguồn trọng số bằng nhau) làm giảm chất lượng ở cả hai bộ: bộ 01  $R@1$  81%  $\rightarrow$  77%,  $R@3$  100%  $\rightarrow$  94%; bộ 02  $R@1$  90%  $\rightarrow$  60%. Vì BM25 một mình chỉ đạt 58%/40% ở  $R@1$ , việc cho nó trọng số ngang với vector kéo các Node khớp từ khóa nhưng sai ngữ nghĩa lên đầu. Để tách nguyên nhân, đo án đo thêm biến thể `relative_score` với trọng số ưu tiên vector 3:1: kết quả *tốt nhất trên bộ 01* ( $R@1$  84%, MRR 0,914, nDCG@1 0,839) nhưng lại *kém hơn vector* trên bộ 02 ( $R@1$  80% so với 90%). Nói cách khác, chênh lệch giữa các biến thể hybrid nằm trong khoảng nhiễu của bộ câu hỏi, còn tham số trộn thì quyết định kết quả nhiều hơn cả việc “có BM25 hay không”. Đây là lý do `use_hybrid` mặc định vẫn là `False`: muốn bật phải chọn fusion mode và trọng số bằng đo trên chính corpus của mình.

**4. Kiểm định thống kê: bộ câu hỏi hiện tại chưa đủ mạnh để khẳng định các chênh lệch nhỏ.** Kiểm định McNemar chính xác trên  $R@1$ :

- vector vs BM25, bộ 01: chỉ vector đúng 9 câu, chỉ BM25 đúng 2 câu,  $p = 0,065$ ;
- vector vs BM25, bộ 02: 5 câu so với 0 câu,  $p = 0,062$ ;
- vector vs hybrid ưu tiên vector, bộ 01: 0 câu so với 1 câu,  $p = 1,000$ .

Như vậy khoảng cách vector–BM25 rất lớn về độ lớn nhưng *chưa đạt* ngưỡng  $p < 0,05$  với 31 câu, còn chênh lệch +3 điểm phần trăm của biến thể hybrid tốt nhất hoàn toàn có thể là may mắn. Đây là hạn chế về *độ mạnh thống kê* của bộ câu hỏi, không phải về hệ thống, và là lý do phải báo cáo khoảng tin cậy thay vì chỉ một con số.

## 8.4 Dừng ở đâu trong đồ án

Cả hai đều có công tắc (`use_hybrid`, `use_rerank`; `--rerank`, `--hybrid` trên dòng lệnh; ô chọn trong giao diện Gradio) và **mặc định tắt** theo kết quả đo. `make eval` đo lại cả bốn cấu hình khi corpus thay đổi.

## CHƯƠNG 9. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

### 9.1 Mô tả bài toán

Sinh viên SGU hỏi bằng tiếng Việt — thường viết tắt, sai chính tả, hỏi nối tiếp — về quy chế đào tạo, và nhận câu trả lời ngắn kèm văn bản và Điều nguồn để tự kiểm tra. Kho tài liệu gồm 6 văn bản đang áp dụng, trong đó có văn bản sửa đổi (QĐ 3183/2025) chỉ áp dụng một phần cho khóa tuyển sinh từ 2025.

### 9.2 User story

Các câu chuyện người dùng dưới đây lấy từ câu hỏi thật trong nhật ký hỏi đáp:

- Là sinh viên, tôi muốn hỏi “thời gian tối đa hoàn thành khóa học” và nhận câu trả lời kèm Điều để tự kiểm tra.
- Là sinh viên, khi hỏi điều không có trong quy chế (“bao nhiêu tiền một tín chỉ”), tôi muốn hệ thống nói rõ là không tìm thấy.
- Là sinh viên khóa 2025, khi hỏi “8,4 hệ 10 thì hệ 4 được bao nhiêu”, tôi muốn câu trả lời theo thang điểm áp dụng cho khóa của tôi.
- Là sinh viên, tôi muốn hỏi bằng câu không dấu hoặc sai chính tả (“xuất sất”, “dạt điều khiện”) mà vẫn được trả lời.
- Là sinh viên, sau khi hỏi về cử nhân, tôi muốn hỏi tiếp “còn kỹ sư?” mà không phải lặp lại cả câu.

Hai story cuối mới được đáp ứng một phần (BM25 bỏ dấu) hoặc chưa (không nhớ hội thoại).

### 9.3 Thiết kế

#### 9.3.1 Sơ đồ pipeline

Pipeline thật gồm sáu giai đoạn (Hình 3.1, Chương 3): dữ liệu → Node → embedding/index → truy hồi → tổng hợp (ngữ cảnh + LLM + trích nguồn) → đánh giá.

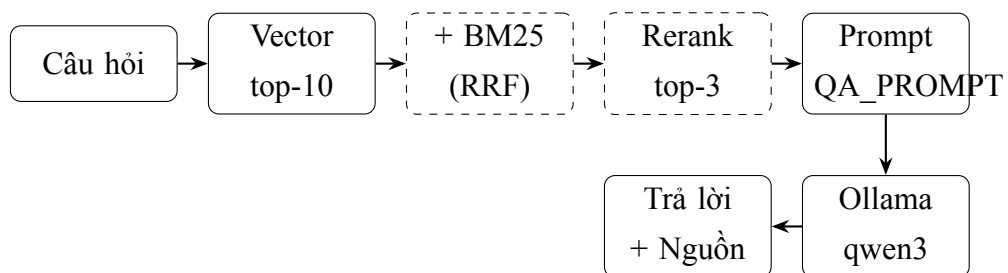
### 9.3.2 Mô hình dữ liệu

| Khóa metadata     | Ý nghĩa                                   | Embed | Prompt |
|-------------------|-------------------------------------------|-------|--------|
| nguồn_trich       | “Điều 9 — Quy chế ...2021” (sinh tự động) | có    | có     |
| van_ban           | Tên hiển thị của văn bản (src/corpus.py)  | không | có     |
| chuong            | Tên Chương                                | không | có     |
| dieu/sua_doi_dieu | Số Điều, hoặc số Điều được sửa đổi        | không | có     |
| phan              | chinh / sua_doi / ban_hanh / noi_dung     | không | không  |
| phan_doan         | Thứ tự đoạn khi một Điều bị cắt tiếp      | không | có     |
| file,file_goc,url | Truy vết về file tầng 2 và PDF gốc        | không | không  |

Bảng 9.1: Metadata của Node (chỉ khóa có giá trị mới được gắn).

Bộ câu hỏi truy hồi là JSON {"question", "dieu\_dung"}; bộ chấm câu trả lời thêm phạm\_vi (“trong”/“ngoài”), dap\_an và nguồn.

### 9.3.3 Luồng truy vấn



Hình 9.1: Luồng truy vấn; khung nét đứt là bước tùy chọn, mặc định tắt.

## CHƯƠNG 10. TRIỂN KHAI, TÍCH HỢP VÀ MINH CHỨNG KẾT QUẢ

### 10.1 Tích hợp hệ thống

- **Dùng trực tiếp:** `VectorStoreIndex`, `StorageContext`, `SentenceSplitter`, `QueryFusionRetriever`, `SentenceTransformerRerank`, `SimilarityPostprocessor`, `RetrieverQueryEngine`, `RetrieverEvaluator`, các evaluator câu trả lời.
- **Viết lại / tự viết:** bước dữ liệu (OCR + Markdown hai tầng thay `SimpleDirectoryReader`), loader Node theo heading, BM25, client model server, độ đo thống kê.
- **Điểm vào:** `src/pipeline.py` (dòng lệnh), `app/app.py` (Gradio, ghi nhật ký câu hỏi), `eval/run_batch_cau_hoi.py` (chạy hàng loạt).
- **Thí nghiệm chỉ để ra quyết định:** so sánh vector/rerank/hybrid (quyết định tất cả hai), chọn 6 văn bản đang áp dụng thay vì nạp cả văn bản hết hiệu lực (văn bản cũ lần át Quy chế 2021 khi truy hồi), chọn profile server làm cấu hình đo chính.

### 10.2 Kết quả đạt được

#### 10.2.1 Truy hồi

Bảng 4.3 và 4.4 (Chương 4) cho chỉ số truy hồi; Bảng 4.2 cho top- $k$  Node của các câu mẫu; Bảng 8.1 (Chương 8) so sánh các cấu hình nâng cao.

#### 10.2.2 Câu trả lời

##### 10.2.2.1 Cách đánh giá

Bản báo cáo trước chấm câu trả lời bằng tay trên một phiên hỏi đáp 56 câu. Bản này bổ sung `eval/evaluate_answers.py` để chấm tự động bằng chính các evaluator của LlamaIndex, chạy hàng loạt qua `BatchEvalRunner`:

- `FaithfulnessEvaluator` — câu trả lời có suy ra được từ ngữ cảnh truy hỏi không;
- `RelevancyEvaluator` — câu trả lời có đúng câu hỏi không;
- `CorrectnessEvaluator` — so với đáp án chuẩn trong `eval/cau_hoi_danh_gia.json`;
- chế độ `closed-book` (LLM trả lời *không* có ngữ cảnh) làm mốc so sánh RAG với không RAG.

Bộ câu hỏi chấm câu trả lời gồm 13 câu, mỗi câu có trường `pham_vi` (“trong” / “ngoài”) và đáp án chuẩn lấy từ bảng đã kiểm chứng thủ công ở bản báo cáo trước. Nhờ trường `pham_vi`, script tách **từ chối đúng** (câu ngoài phạm vi) khỏi **từ chối sai** (đáp án có trong ngữ cảnh nhưng hệ thống vẫn nói không tìm thấy) — chính là hai loại đã bị gộp thành một con số “52% từ chối” ở bản trước.

**Trạng thái chạy.** Script đã viết xong, kiểm thử import và logic nhận dạng từ chối, nhưng **chưa chạy** `make eval-answers`. Trong tuần 05–09/10, LLM đã chạy được qua Ollama trên Colab (`scripts/setup_server_v2.sh`), và nhóm dùng nó để chạy lại 56 câu hỏi của phiên trước với prompt mới (Mục 10.2.2.4) — vẫn chấm tay. Vì vậy Mục 10.2.2.2 trình bày kết quả chấm tay của phiên chạy trước (có ghi rõ nguồn), còn các bảng điểm `faithfulness/relevancy/correctness` sẽ được điền khi chạy `make eval-answers`.

#### 10.2.2.2 Kết quả chấm tay (phiên chạy 56 câu, profile **server** có rerank)

Nhật ký `eval/results/test-basic.md` ghi một phiên chạy `src/pipeline.py`: 56 câu hỏi do nhóm tự đặt theo cách sinh viên thật sự hỏi, gồm cả viết tắt (“bao nhiêu chỉ”), sai chính tả (“xuất sắt”), câu hỏi nối tiếp (“còn kỹ sư”) và câu ngoài phạm vi. Hệ thống trả lời “không tìm thấy” ở 29/56 câu (52%) và đưa ra câu trả lời ở 27 câu.

| Câu hỏi                                                  | Hệ thống trả lời                              | Quy chế 2021 quy định                                                                                                                   | Đánh giá     |
|----------------------------------------------------------|-----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|--------------|
| Học cử nhân cần bao nhiêu tín chỉ                        | Tối thiểu 120 tín chỉ                         | 120 tín chỉ (Điều 2)                                                                                                                    | Đúng         |
| Muốn được B cần bao nhiêu điểm                           | Từ 7,0 đến 8,4                                | B: 7,0–8,4 (Điều 9)                                                                                                                     | Đúng         |
| Cảnh báo kỳ 1 và 3, kỳ 2 bình thường có bị buộc thôi học | Không, vì hai lần không liên tiếp             | Buộc thôi học khi quá hai lần cảnh báo <i>liên tiếp</i> (Điều 11)                                                                       | Đúng         |
| Mua bằng tiếng Anh giả nộp cho trường                    | Không được; buộc thôi học, thu hồi bằng       | Như câu trả lời (Điều 19)                                                                                                               | Đúng         |
| 8,4 hệ 10 thì hệ 4 được bao nhiêu                        | 3,4 điểm                                      | Khóa trước 2025: 8,4 là B, quy đổi 3,0; khóa 2025 trở đi (QĐ 3183): 8,4 là B+, quy đổi 3,5 (Điều 9, 10)                                 | Sai          |
| Khối lượng học kỳ chính                                  | Tối thiểu 30 tuần                             | 2 học kỳ chính <i>cộng lại</i> tối thiểu 30 tuần (Điều 6)                                                                               | Sai          |
| Học bị F thì làm gì                                      | Học lại; điểm lần học cuối là điểm chính thức | Học phần không đạt phải học lại, điểm <i>lần học cuối</i> là điểm chính thức; “cao nhất” chỉ áp dụng khi học cải thiện (Điều 9 khoản 4) | Đúng*        |
| Học điểm P thì sao                                       | P không tính vào điểm TB, “quy đổi thành 0”   | P không được tính vào điểm TB (Điều 10 khoản 3), không quy đổi                                                                          | Sai một phần |
| Ra trường xuất sắc cần bao nhiêu điểm                    | Không tìm thấy                                | Từ 3,6 đến 4,0 (Điều 10), và Điều 10 có trong 3 Điều nguồn                                                                              | Từ chối sai  |
| Còn kỹ sư (hỏi tiếp câu cử nhân)                         | Không tìm thấy                                | 150 tín chỉ; hỏi lại đầy đủ thì trả lời đúng                                                                                            | Từ chối sai  |
| Điểm rèn luyện là gì                                     | Không tìm thấy                                | Không thuộc Quy chế đào tạo (văn bản riêng của CTSV)                                                                                    | Từ chối đúng |
| Bao nhiêu tiền một tín chỉ                               | Không tìm thấy                                | Không thuộc Quy chế đào tạo                                                                                                             | Từ chối đúng |

Bảng 10.1: Một số câu trả lời trong nhật ký phiên chạy trước, đối chiếu với văn bản Quy chế 2021. Đây là số liệu chấm tay, không sinh lại được bằng lệnh. \*Bản báo cáo trước chấm câu này là “Sai một phần”; khi đối chiếu lại khoản 4 Điều 9 trong tuần 05–09/10 thì câu trả lời của hệ thống là đúng, còn đáp án tham chiếu đã nhầm hai trường hợp học lại và học cải thiện.

### 10.2.2.3 Phân tích

**Prompt chống bịa hoạt động, nhưng hơi quá tay.** Với câu hỏi ngoài phạm vi, hệ thống từ chối thay vì bịa — đúng mục tiêu. Nhưng tỷ lệ từ chối 52% cao hơn tỷ lệ câu thật sự ngoài phạm vi: có câu mà thông tin *đã nằm trong ngữ cảnh* vẫn bị từ chối, như câu hỏi về điểm ra trường loại xuất sắc (viết sai chính tả “xuất sất”). Đây chính là lý do phải tách “từ chối đúng” khỏi “từ chối sai” như thiết kế ở Mục 10.2.2.

**Lỗi sai tập trung ở phép tính và chi tiết số.** Các câu trả lời sai đều có Điều đúng trong ngữ cảnh (truy hồi đúng), nhưng LLM đọc sai hoặc suy diễn thêm: tự “quy đổi” 8,4 thành 3,4 thay vì tra bảng, hay đọc “30 tuần cho 2 học kỳ” thành “30 tuần mỗi học kỳ”. Đây là loại lỗi nguy hiểm nhất vì câu trả lời vẫn trích dẫn đúng Điều. Bản này bổ sung hai ràng buộc trong QA\_PROMPT nhằm đúng loại lỗi này: *trích nguyên văn câu quy định trước khi kết luận và không tự quy đổi thang điểm, cần tra bảng thì chép đúng hàng của bảng*. Hiệu quả của hai ràng buộc này cần được đo bằng `make eval-answers` trên máy có LLM, và được ghi nhận là việc còn lại.

**Câu trả lời không ổn định — đã xử lý ở mức mã nguồn.** Bản trước ghi nhận cùng một câu hỏi chỉ khác một dấu hỏi chấm mà lần đầu từ chối, lần sau trả lời “Không”. Nguyên nhân là Ollama (...) không đặt `temperature`, nên mô hình lấy mẫu ngẫu nhiên. Bản này đặt `temperature=0` và `seed=42` trong `make_llm()`; đây là sửa đổi đã kiểm tra ở mức mã nguồn nhưng cần một phiên chạy LLM mới để xác nhận bằng số.

**Không có ngữ cảnh hội thoại.** `RetrieverQueryEngine` gửi từng câu hỏi độc lập, nên câu nối tiếp “còn kỹ sư” bị từ chối. Sinh viên thường hỏi nối tiếp, nên đây vẫn là hạn chế lớn nhất ở tầng sinh.

### 10.2.2.4 Chạy lại 56 câu với prompt mới (tuần 05–09/10)

**Cách chạy.** Cùng 56 câu hỏi của phiên trước (`eval/results/cau-hoi01.md`) được chạy lại bằng `eval/run_batch_cau_hoi.py` (bản notebook: `notebooks/chay_cau_hoi01.ipynb`), profile server, qwen3:8b trên Colab, `temperature=0`, `seed=42`. Kết quả ghi ở `eval/results/cau-hoi01-result.md` (phiên bản trong commit `be7b81e`, ngày 08/10). Prompt dùng

trong lần chạy này đã có hai ràng buộc “trích nguyên văn” và “không tự quy đổi” nhưng *chưa* có chú giải thang điểm ở Mục 4.5.2.

**Lưu ý khi so sánh.** Hai phiên khác nhau ở *hai* điểm cùng lúc: prompt mới, và cấu hình ngữ cảnh — phiên trước bật rerank (LLM chỉ nhận 3 Node), phiên này tắt rerank (LLM nhận 10 Node). Vì vậy chênh lệch dưới đây không quy được riêng cho prompt; muốn tách phải chạy thêm một phiên chỉ đổi một yếu tố.

**Kết quả tổng.** Số câu trả lời “không tìm thấy” giảm từ 29/56 (52%) xuống 23/56 (41%).

| Câu hỏi                                                         | Trả lời (phiên 08/10)                                                          | Quy chế quy định                                                    | So với phiên trước                   |
|-----------------------------------------------------------------|--------------------------------------------------------------------------------|---------------------------------------------------------------------|--------------------------------------|
| Ra trường xuất sắc cần bao nhiêu điểm                           | ĐTB tích lũy 3,6–4,0                                                           | 3,6–4,0 (Điều 10 khoản 4)                                           | Từ chối sai → <b>Đúng</b>            |
| 8,4 hệ 10 thì hệ 4 được bao nhiêu                               | 3,5                                                                            | 3,5 với khóa 2025 trở đi (B+); 3,0 với khóa trước                   | Sai → Đúng một phần (không nêu khóa) |
| Điểm D+ ứng với khoảng nào trên thang 10                        | 4,8–5,4                                                                        | 4,8–5,4 (QĐ 3183, Điều 1 khoản 3)                                   | (câu mới) Đúng                       |
| Giỏi tiếng Anh nhưng không có chứng chỉ có ra trường được không | Được xét nếu đủ điều kiện; chưa đạt chuẩn ngoại ngữ thì có 3 năm để hoàn thiện | Như câu trả lời (Điều 13 khoản 4)                                   | Đúng                                 |
| Không có chứng chỉ tiếng Anh có ra trường được không            | Không, dẫn Điều 3 về học chương trình thứ hai ngành Ngôn ngữ Anh               | Điều 13 khoản 4, như câu trên                                       | <b>Sai</b> (lấy nhầm văn bản QĐ 810) |
| Học bị F thì làm gì                                             | F quy đổi 0 và “không được tính vào điểm trung bình”                           | F quy đổi 0 và <i>được</i> tính (Điều 10 khoản 2–3)                 | Đúng → <b>Sai</b>                    |
| Khối lượng học kỳ chính                                         | Mỗi học kỳ chính tối thiểu 30 tuần                                             | 2 học kỳ chính cộng lại tối thiểu 30 tuần (Điều 6)                  | Sai (không đổi)                      |
| Rớt một môn, các kỳ còn lại 4.0 có được xuất sắc không          | Không tìm thấy                                                                 | Chỉ bị giảm hạng nếu học lại vượt 5% tổng tín chỉ (Điều 13 khoản 3) | Từ chối sai                          |
| Còn kỹ sư (hỏi tiếp)                                            | Không tìm thấy                                                                 | 150 tín chỉ                                                         | Từ chối sai (không đổi)              |
| Mua bằng tiếng Anh giả (hai cách hỏi)                           | Một cách hỏi: đúng, dẫn Điều 19 khoản 3; cách kia: không tìm thấy              | Buộc thôi học, thu hồi bằng (Điều 19 khoản 3)                       | Không ổn định theo cách diễn đạt     |
| Bao nhiêu tiền 01 tín                                           | In ra nguyên chuỗi / no_think                                                  | Ngoài phạm vi                                                       | Từ chối đúng → <b>Lỗi</b>            |

Bảng 10.2: Một số câu của phiên chạy lại ngày 08/10, chấm tay theo văn bản trong data/processed.

## Phân tích.

- **Ít từ chối sai hơn**, rõ nhất ở câu “xuất sắc” — trước đây bị từ chối dù Điều 10 có trong ngữ cảnh. Do ngữ cảnh tăng từ 3 lên 10 Node cùng lúc với đôi prompt, chưa kết luận được yếu tố nào có tác dụng.
- **Phạm vi hiệu lực theo khóa tuyển sinh là nguồn sai mới.** QĐ 3183 chỉ áp dụng thang điểm có B+, C+, D+ cho khóa tuyển sinh từ 2025 (Điều 2 của QĐ). Hệ thống trả lời theo một trong hai bảng mà không nói bảng đó áp dụng cho khóa nào: “8,4 → 3,5” đúng với khóa 2025, còn “B là 7,0–8,4” chỉ đúng với khóa trước 2025. Prompt hiện chỉ nói “có văn bản sửa đổi thì dùng nội dung đã sửa đổi”, tức là ngầm giả định mọi sinh viên đều thuộc khóa mới.
- **Ngữ cảnh rộng hơn cũng kéo theo văn bản nhiều.** Câu “không có chứng chỉ tiếng Anh” lấy Điều 3 của QĐ 810 (điều kiện học chương trình thứ hai ngành Ngôn ngữ Anh) làm căn cứ, trong khi Điều 13 — căn cứ đúng — cũng có trong ngữ cảnh. Câu về điểm F sai ở chi tiết “không tính vào điểm trung bình”, nhầm F với các điểm chữ không quy đổi ở khoản 3 Điều 10.
- **Không ổn định theo cách diễn đạt.** `temperature=0` làm *cùng một* câu hỏi cho cùng câu trả lời, nhưng hai cách hỏi gần nhau vẫn có thể cho một câu đúng và một câu “không tìm thấy”. Tính tắt định không thay được độ vững theo diễn đạt.
- **Rò chỉ thị /no\_think.** Ở một câu ngoài phạm vi, Qwen3 trả về đúng chuỗi điều khiển `/no_think` thay cho câu trả lời. Cần lọc chuỗi này ở đầu ra và đếm nó như một lỗi trong `evaluate_answers.py`.

### 10.2.3 So sánh trước – sau các tối ưu

| Vấn đề                             | Trước                                | Sau                                        |
|------------------------------------|--------------------------------------|--------------------------------------------|
| Điều vắt qua hai trang             | Bị chia đôi (mỗi trang một Document) | Một Node mỗi Điều (Markdown hai tầng)      |
| Điều 1–3 của quyết định ban hành   | Trùng nhãn với Điều 1–3 của quy chế  | Tách phân="ban_hanh"                       |
| Metadata rỗng trong embedding      | Có (chuong:, dieu: rỗng)             | Chỉ nguồn_trich                            |
| Dùng nhằm chỉ mục cũ               | Có thể, nếu quên --rebuild           | Fingerprint tự build lại                   |
| Câu trả lời giữa các lần hỏi       | Không ổn định                        | temperature=0, seed=42                     |
| Embedding + reranker trên GPU 4 GB | CUDA out of memory                   | fp16, khoảng 2,2 GB                        |
| Tỷ lệ “không tìm thấy” (56 câu)    | 52%                                  | 41% (đổi prompt và top- <i>k</i> cùng lúc) |

Bảng 10.3: Các thay đổi đã đo hoặc kiểm chứng được.

### 10.2.4 Hướng dẫn chạy lại

Đoạn mã 10.1: Các lệnh chạy lại (từ thư mục gốc repo).

```
1 python3 -m venv venv && venv/bin/pip install -r requirements.txt
2 bash scripts/pull_models.sh server # hoặc bash
   scripts/setup_server_v2.sh trên Colab
3 RAG_PROFILE=server venv/bin/python src/index_builder.py
4 RAG_PROFILE=server venv/bin/python src/pipeline.py # hỏi đáp dòng
   lệnh
5 RAG_PROFILE=server venv/bin/python app/app.py # giao diện Gradio
6 make test && make eval && make baseline # kiểm thử và số liệu
   truy hồi
```

## 10.3 Bảng phủ kỹ thuật

Bảng phản ánh trạng thái tuần 5; bản cuối sẽ khóa cùng commit niêm phong, nộp ít nhất 24 giờ trước buổi bảo vệ. Chỉ khai những gì có trong commit đó.

| TT | Nội dung                    | Tầng  | Chức năng       | File                                  | Minh chứng           | Trạng thái         |
|----|-----------------------------|-------|-----------------|---------------------------------------|----------------------|--------------------|
| 1  | OCR + Markdown hai tầng     | 1A    | Nạp quy chế     | scripts/extract_to_md.py, clean_md.py | 6 văn bản            | Dùng               |
| 2  | Node theo Điều              | 1A+1B | Đơn vị truy hồi | src/processed_loader.py               | 54 Node, kiểm thử    | Dùng               |
| 3  | Embedding + index + persist | 1A+1B | Lập chỉ mục     | src/index_builder.py                  | 77 s / 1,5 s         | Dùng               |
| 4  | Retriever top- $k$          | 1A    | Truy hồi        | src/retriever.py                      | Bảng 4.2             | Dùng               |
| 5  | Query engine + trích nguồn  | 1A+1B | Hỏi đáp         | src/query_engine.py                   | Dòng “Nguồn”         | Dùng               |
| 6  | Recall@ $k$ , MRR, nDCG, CI | 1A    | Kiểm chứng      | eval/evaluate.py, metrics.py          | R@1 81%, R@3 100%    | Dùng               |
| 7  | Từ chối ngoài corpus        | 1B    | Hỏi đáp         | src/query_engine.py                   | 41% “không tìm thấy” | Chưa đo tỷ lệ đúng |
| 8  | Hybrid BM25                 | 3     | Truy hồi        | src/bm25.py                           | Bảng 8.1             | Có, tắt            |
| 9  | Rerank cross-encoder        | 3     | Truy hồi        | src/retriever.py                      | Bảng 8.1             | Có, tắt            |
| 10 | Model server HTTP           | 3     | Hạ tầng         | src/model_server.py                   | Chạy trên Colab      | Dùng               |
| 11 | .env, 2 profile             | 2     | Cấu hình        | src/config.py, .env.example           | —                    | Dùng               |
| 12 | Unit test + CI              | 2     | Kiểm thử        | tests/, .github/workflows/ci.yml      | 26 kiểm thử          | Dùng               |
| 13 | Giao diện Gradio            | 1B    | Demo            | app/app.py                            | Nhật ký câu hỏi      | Chạy thử           |

Bảng 10.4: Bảng phủ kỹ thuật.

## CHƯƠNG 11. ĐÁNH GIÁ VÀ NHÌN LẠI

### 11.1 So sánh với mục tiêu ban đầu

| Mục tiêu (Lời mở đầu)                             | Kết quả         | Minh chứng         |
|---------------------------------------------------|-----------------|--------------------|
| 6 mục lỗi Tầng 1A                                 | Đạt 6/6         | Bảng 4.1           |
| Custom node parser, persistent index, trích nguồn | Đạt             | Chương 5–7         |
| Từ chối câu ngoài corpus                          | Một phần        | Chưa đo tỷ lệ đúng |
| Thí nghiệm chunking                               | Chưa            | Mục 4.2            |
| Hybrid, rerank (Tầng 3)                           | Đạt, có số liệu | Chương 8           |
| Kiểm thử + CI (Tầng 2)                            | Đạt             | 26 kiểm thử        |
| Chăm câu trả lời tự động                          | Chưa chạy       | Mục 10.2.2         |

Bảng 11.1: Đối chiếu mục tiêu và kết quả.

#### Các quyết định đổi hướng.

- **Từ OCR theo trang sang Markdown hai tầng:** vì Điều bị chia đôi giữa hai trang và Điều của quyết định ban hành trùng nhau (Chương 5).
- **Từ một văn bản sang sáu văn bản đang áp dụng:** để trả lời được nội dung sửa đổi; văn bản hết hiệu lực bị loại vì lần át Quy chế 2021 khi truy hỏi.
- **Chọn profile `server` làm cấu hình chính, LLM chạy từ xa:** GPU 4 GB không chạy nổi `qwen3:8b` cùng embedding; LLM chạy trên máy GPU thuê hoặc Colab (`scripts/setup_server_v2.sh`), embedding chạy local ở fp16 hoặc CPU.
- **Tắt rerank và hybrid:** đo thấy không giúp ở quy mô 54 Node (Chương 8).

### 11.2 Hạn chế

- **Độ mạnh thống kê của bộ câu hỏi.**  $31 + 10 = 41$  câu là ít; khoảng tin cậy rộng (bộ 02:  $R@1$  90% với CI 70–100%) và kiểm định vector-BM25 chưa đạt  $p < 0,05$ . Cần mở rộng lên khoảng 150–300 câu, phân tầng theo loại câu hỏi, và có hai người gán nhãn độc lập để báo cáo độ đồng thuận.

- **Corpus nhỏ.** 6 văn bản / 54 Node. Kết luận “rerank và hybrid không giúp” chỉ đúng ở quy mô này; ở kho vài trăm Node trở lên, hai kỹ thuật đó cần được đo lại.
- **Chất lượng OCR.** Văn bản quan trọng nhất (QĐ 3183/2025) có chất lượng OCR thấp nhất (94,9% âm tiết hợp lệ) và vẫn còn lỗi như “Sửa đổi”.
- **Metadata hiệu lực còn trống.** `_metadata_van_ban.csv` chưa có số hiệu, ngày ban hành, tình trạng hiệu lực, nên chưa áp dụng được lọc theo hiệu lực hay theo khóa.
- **Đánh giá câu trả lời chưa chạy tự động.** Script đã có và LLM đã chạy được trên Colab, nhưng `make eval-answers` chưa chạy; câu trả lời mới chỉ được chấm tay.
- **Hạ tầng đo.** Số liệu trong chương này đo trên CPU; thời gian tuyệt đối không so sánh được với các con số trên GPU ở bản báo cáo trước.
- **Tầng sinh.** Không nhớ hội thoại, không có công cụ tra bảng/quy đổi điểm, chỉ một lượt gọi LLM trên ngữ cảnh đã truy hồi.
- **Không phân biệt khóa tuyển sinh.** Thang điểm của QĐ 3183 chỉ áp dụng từ khóa 2025, nhưng hệ thống không hỏi hay nêu khóa nên câu trả lời về điểm chỉ đúng với một nhóm sinh viên (Mục 10.2.2.4).
- **Chú giải thang điểm trong prompt chưa được đo.** Đây là kiến thức riêng của quy chế SGU đặt trong prompt (Mục 4.5.2), và chưa có phiên chạy nào xác nhận nó sửa được lỗi.

## 11.3 Phân tích lỗi

### 11.3.1 Lỗi 1: LLM nhầm thang điểm hệ 10 với hệ 4

- **Hiện tượng:** “A thì được bao nhiêu điểm hệ 10” → “A được quy đổi thành 4,0 điểm hệ 10” (nhật ký `questions_log.jsonl`; top-*k* (Bảng 4.2) cho thấy truy hồi đúng, lỗi nằm ở tầng sinh).
- **Phát hiện:** đọc nhật ký `questions_log.jsonl`; top-*k* (Bảng 4.2) cho thấy truy hồi đúng, lỗi nằm ở tầng sinh.
- **Nguyên nhân và cách sửa:** Mục 4.5.2. Chưa đo lại sau khi sửa.

### 11.3.2 Lỗi 2: CUDA out of memory khi nạp embedding cùng reranker

- **Hiện tượng:** trên card 4 GB, profile server chiếm khoảng 3,5 GB sau khi nạp embedding và báo hết bộ nhớ khi nạp reranker.
- **Nguyên nhân:** SentenceTransformerRerank không nhận dtype, luôn nạp fp32.
- **Cách sửa:** nạp reranker trên CPU, ép fp16 rồi mới chuyển lên GPU:

Đoạn mã 11.1: Nạp reranker fp16 (make\_local\_reranker() trong src/retriever.py).

```
1 reranker = SentenceTransformerRerank(model=cfg.reranker_model_name,  
    top_n=cfg.reranker_top_n,  
2                                     device="cpu" if fp16 else device)  
3 if fp16:  
4     reranker._model.half().to(device)  
5     reranker.device = device
```

Sau khi sửa, cả hai mô hình cùng nằm trên GPU, tổng khoảng 2,2 GB (Mục 3.1.4).

## 11.4 AI Disclosure

*(Chưa làm — phần này viết ở bản báo cáo cuối, dựa trên nhật ký dùng AI của nhóm).*

Nội dung sẽ gồm:

1. Công cụ AI đã dùng và dùng vào việc gì.
2. Phần có tỷ trọng AI sinh cao (ghi file / chức năng).
3. Ít nhất hai tình huống AI trả kết quả *sai*: prompt, AI trả về gì, sai ở đâu, phát hiện thế nào, sửa ra sao.
4. Tự đánh giá: nếu bỏ AI, phần nào nhóm vẫn tự viết lại được.

## CHƯƠNG 12. HƯỚNG PHÁT TRIỂN

### 12.1 Cải tiến sản phẩm

**Đã xong trong bản này** (so với danh sách “hướng phát triển” của bản báo cáo trước): tách thư mục chỉ mục theo profile; fp16 cho embedding và reranker; model server HTTP cho embedding/reranker; mở rộng lên 6 văn bản; giao diện Gradio; ghim phiên bản thư viện; kiểm thử tự động; đánh giá có khoảng tin cậy, baseline và kiểm định cặp; câu trả lời tắt định (`temperature=0`, `seed`). Trong tuần 05–09/10: chạy LLM và model server trên Colab qua Cloudflare tunnel; ghi nhật ký câu hỏi của giao diện Gradio; chạy lại 56 câu với prompt mới; thêm chú giải thang điểm vào prompt và công tắc RAG\_THINKING.

**Còn lại, theo thứ tự ưu tiên:**

1. **Chạy** `make eval-answers` (LLM trên Colab) để có số `faithfulness/relevancy/correctness`, và so chế độ RAG với `closed-book`. Đây là việc còn thiếu duy nhất của phần đánh giá. Cùng đợt chạy đó, chạy lại 56 câu theo từng yếu tố một (chỉ đổi prompt, chỉ đổi `top-k` đưa vào LLM, bật/tắt RAG\_THINKING) để tách tác dụng của từng thay đổi ở Mục 10.2.2.4.
2. **Xử lý khóa tuyển sinh**: gắn metadata phạm vi áp dụng (ví dụ “khóa 2025 trở đi”) cho mục sửa đổi của QĐ 3183, và yêu cầu câu trả lời về điểm nêu rõ áp dụng cho khóa nào (hoặc nêu cả hai bảng). Cùng lúc, gắn câu dẫn “thang điểm 10”/“quy đổi để tính điểm trung bình” vào mục sửa đổi để thay chú giải đang đặt cứng trong prompt.
3. **Mở rộng bộ câu hỏi** lên vài trăm câu, có nhãn loại câu (trong/ngoài phạm vi, viết tắt, nối tiếp, cần tra bảng) và hai người gán nhãn độc lập.
4. **Điền metadata hiệu lực** (`so_hieu`, `ngay_ban_hanh`, `trinh_trang`) rồi dùng `MetadataFilters` để loại văn bản hết hiệu lực — `processed_loader` đã sẵn sàng nhận các trường này từ front matter.
5. **Liên kết Điều gốc với mục sửa đổi** bằng `NodeRelationship`: khi truy hỏi được Điều 7 của Quy chế 2021 thì tự động kèm mục “sửa đổi Điều 7” của QĐ 3183, và ngược lại.

6. **Small-to-big**: cắt thêm Node con theo khoản, truy hồi trên Node con rồi đưa Điều cha cho LLM (`HierarchicalNodeParser + AutoMergingRetriever`) — nhằm đúng lỗi “đọc sai chi tiết số” trong Điều dài.
7. **Chat engine có ngữ cảnh** (`CondensePlusContextChatEngine`) cho câu hỏi nối tiếp; giao diện Gradio đã có sẵn lịch sử chat.
8. **Công cụ tra bảng quy đổi điểm** (`FunctionTool + agent`) thay vì để LLM tự tính.
9. **Vector store ngoài** (Chroma/Qdrant) khi corpus vượt vài trăm Node, để lọc meta-data phức tạp và cập nhật từng phần.

## 12.2 Định hướng bản thân

*(Chưa làm — từng thành viên viết ở bản báo cáo cuối).*

## 12.3 Kết luận

Trên corpus 6 văn bản / 54 Node, hệ thống đạt  $\text{Recall}@1 = 81\%$  và  $\text{Recall}@3 = 100\%$  ở bộ 31 câu hỏi về Quy chế 2021, và  $\text{Recall}@1 = 90\%$  ở bộ 10 câu về nội dung sửa đổi — trong khi baseline tìm từ khóa BM25 chỉ đạt 58% và 40% ở  $\text{R}@1$ . Chỉ số do hệ thống tự tính khớp với chỉ số do `RetrieverEvaluator` của `LlamaIndex` tính. Hai kỹ thuật “nâng cao” thường được khuyến nghị — xếp hạng lại bằng cross-encoder và hybrid BM25 + vector — *không* cải thiện trên quy mô này và có thể làm giảm chất lượng nếu chọn sai tham số trộn, nên cả hai đều mặc định tắt và có công tắc đo lại bằng `make eval`. Điểm yếu hiện nay không nằm ở truy hồi vector mà ở ba chỗ: dữ liệu (54 Node, metadata hiệu lực trống), tăng sinh (LLM đọc sai số liệu, không nhớ hội thoại), và độ mạnh thống kê của bộ câu hỏi đánh giá.

## TÀI LIỆU THAM KHẢO

- [1] LlamaIndex Team. *LlamaIndex Documentation (phiên bản 0.14.24)*. <https://docs.llamaindex.ai/>. Truy cập tháng 10/2026. 2025.
- [2] LlamaIndex Team. *run-llama/llama\_index: Mã nguồn thư viện*. [https://github.com/run-llama/llama\\_index](https://github.com/run-llama/llama_index). Phiên bản đo trong báo cáo: 0.14.24. 2025.
- [3] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2020.
- [4] Yunfan Gao et al. “Retrieval-Augmented Generation for Large Language Models: A Survey”. In: *arXiv preprint arXiv:2312.10997* (2023).
- [5] Nils Reimers and Iryna Gurevych. “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks”. In: *Proceedings of EMNLP-IJCNLP*. 2019.
- [6] Niklas Muennighoff et al. “MTEB: Massive Text Embedding Benchmark”. In: *arXiv preprint arXiv:2210.07316* (2022).
- [7] Stephen Robertson and Hugo Zaragoza. “The Probabilistic Relevance Framework: BM25 and Beyond”. In: *Foundations and Trends in Information Retrieval* 3.4 (2009), pp. 333–389.
- [8] Nelson F. Liu et al. “Lost in the Middle: How Language Models Use Long Contexts”. In: *Transactions of the Association for Computational Linguistics (TACL)* (2024).
- [9] Vladimir Karpukhin et al. “Dense Passage Retrieval for Open-Domain Question Answering”. In: *Proceedings of EMNLP*. 2020.
- [10] Ollama. *Ollama: chạy mô hình ngôn ngữ lớn cục bộ*. <https://ollama.com/>. 2025.
- [11] Qwen Team. *Qwen3 Technical Report*. <https://qwenlm.github.io/blog/qwen3/>. 2025.
- [12] Tesseract OCR. *Tesseract Open Source OCR Engine*. <https://github.com/tesseract-ocr/tesseract>. 2024.

- [13] BKAI Foundation Models. *bkai-foundation-models/vietnamese-bi-encoder (model card)*. <https://huggingface.co/bkai-foundation-models/vietnamese-bi-encoder>. 2023.
- [14] AITeamVN. *AITeamVN/Vietnamese\_Embedding (model card)*. [https://huggingface.co/AITeamVN/Vietnamese\\_Embedding](https://huggingface.co/AITeamVN/Vietnamese_Embedding). 2024.
- [15] Nandan Thakur et al. “BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models”. In: *NeurIPS Datasets and Benchmarks Track*. 2021.
- [16] Rodrigo Nogueira and Kyunghyun Cho. “Passage Re-ranking with BERT”. In: *arXiv preprint arXiv:1901.04085* (2019).

## PHỤ LỤC A. NHẬT KÝ CÔNG VIỆC HÀNG TUẦN

Mỗi dòng phải khớp với commit trong repo. Cột “Người làm” lấy theo tác giả commit.

| Tuần            | Công việc đã làm                                                                                                                                                                                                               | Người làm                                                                | Khó khăn / kết quả                                                                                                                                  |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| 1–2             | Lập nhóm, chọn công nghệ; cài môi trường, Hello World                                                                                                                                                                          | —                                                                        | Chưa có commit trong repo                                                                                                                           |
| 3 (21–27/09)    | Khởi tạo repo, dữ liệu Quy chế 2021; <code>node_parser.py</code> tách theo Điều; sửa <code>check_model_memory.py</code> ; bộ câu hỏi 01 và 02; dữ liệu sở tay sinh viên; nháp báo cáo (PR #1–#6)                               | Phan Hữu Kha, Võ Thị Như Ý, Trần Hà Thanh Thùy, Huỳnh Thanh Tiến, solozy | Pipeline một văn bản chạy được                                                                                                                      |
| 4 (28/09–04/10) | Cấu hình 2 profile, RAG cơ bản, tiền xử lý Markdown hai tầng, <code>processed_loader.py</code> , fingerprint chỉ mục, model server; báo cáo tuần 4                                                                             | solozy                                                                   | 6 văn bản, 54 Node                                                                                                                                  |
| 5 (05–09/10)    | Chạy LLM + model server trên Colab; hybrid BM25, baseline, kiểm định thống kê; 26 kiểm thử + CI; Gradio có nhật ký câu hỏi; chạy lại 56 câu; chú giải thang điểm trong prompt, <code>RAG_THINKING</code> ; cập nhật Chương 3–5 | solozy                                                                   | Truy hồi R@1 81% / 90%; rerank, hybrid không giúp; “không tìm thấy” 52% → 41%; phát hiện lỗi nhầm hệ 10/hệ 4 và lỗi không phân biệt khóa tuyển sinh |

Bảng A.1: Nhật ký công việc theo tuần, lập từ `git log` (tuần 5: commit `be7b81e` ngày 08/10 và các thay đổi chưa commit). Tên tài khoản `git solozy` do nhóm đối chiếu với thành viên.

## A.1 Tuần 5 (05–09/10/2026)

### A.1.1 Công việc đã làm

- **Hạ tầng:** `scripts/setup_server_v2.sh`, `scripts/colab_server.py` — chạy Ollama và model server trên Colab, mở URL công khai qua Cloudflare tunnel, in sẵn các dòng `.env` cho máy local.
- **Truy hồi:** hybrid BM25 + vector (`src/bm25.py`, `QueryFusionRetriever`); đo bốn cấu hình và biến thể trọng số 3:1; baseline BM25 / không xếp hạng; khoảng tin cậy bootstrap và kiểm định McNemar (`eval/metrics.py`).
- **Kiểm thử:** 26 kiểm thử offline trong `tests/`, CI GitHub Actions, Makefile.
- **Giao diện:** Gradio (`app/app.py`) ghi mỗi câu hỏi vào `eval/results/questions_log.jsonl`.
- **Đánh giá câu trả lời:** chạy lại 56 câu bằng `eval/run_batch_cau_hoi.py` với prompt mới, chấm tay (Mục 10.2.2.4).
- **Tăng sinh:** thêm chú giải thang điểm hệ 10 / hệ 4 vào `QA_PROMPT`; thêm công tắc `RAG_THINKING` (Mục 4.5.2).
- **Báo cáo:** cập nhật Chương 3–5 và phụ lục này.

### A.1.2 Kết quả và khó khăn

- Truy hồi (vector): bộ 01  $R@1$  81%,  $R@3$  100%; bộ 02  $R@1$  90%. Đo lại ngày 08/10 cho cùng chỉ số.
- Rerank và hybrid không cải thiện trên 54 Node, nên để mặc định tắt (Chương 8).
- Phiên chạy lại 56 câu: tỷ lệ “không tìm thấy” giảm từ 52% xuống 41%, nhưng vẫn sai ở số tuần của học kỳ chính, điểm F và câu hỏi về chứng chỉ tiếng Anh; một câu trả về chuỗi `/no_think`.
- Lỗi mới phát hiện: nhầm thang điểm 10 với thang điểm 4 (“A được quy đổi thành 4,0 điểm hệ 10”), và câu trả lời về điểm không phân biệt khóa tuyển sinh trước 2025 với từ 2025 trở đi (QĐ 3183).
- Khi đối chiếu lại khoản 4 Điều 9, nhóm phát hiện đáp án tham chiếu cũ của câu “học bị F” bị nhầm; đã sửa trong Bảng 10.1.

### A.1.3 Việc chuyển sang tuần 6

1. Chạy `make eval-answers` trên LLM Colab; chạy lại 56 câu theo từng yếu tố (prompt, top- $k$  đưa vào LLM, RAG\_THINKING) để đo tác dụng của chú giải thang điểm.
2. Thêm metadata khóa áp dụng cho mục sửa đổi của QĐ 3183 và yêu cầu câu trả lời nêu khóa.
3. Lọc chuỗi `/no_think` khỏi đầu ra và đếm như một lỗi khi chấm.
4. Nạp văn bản quy đổi chứng chỉ tiếng Anh (thư mục `2_quy_doi_tiemg_anh`): câu “quy định về tiếng anh đầu ra” trong nhật ký Gradio hiện trả lời “không tìm thấy”.

## PHỤ LỤC B. BẢNG TỰ KIỂM NHANH BA TẦNG

| Tầng | Nội dung       | Nhóm đã đạt                                                                                                                                                              |
|------|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1A   | Lỗi LlamaIndex | 6 / 6 mục (Bảng 4.1)                                                                                                                                                     |
| 1B   | Tự chọn        | Custom node parser theo Điều; persistent index có fingerprint; trích dẫn nguồn; nhiều nguồn (6 văn bản); từ chối ngoài corpus (một phần, chưa đo tỷ lệ đúng)             |
| 2    | Chuẩn kỹ nghệ  | .env/.env.example, ghim phiên bản requirements.txt, 26 kiểm thử offline, CI GitHub Actions, Makefile, README                                                             |
| 3    | Nâng cao       | Hybrid BM25 + vector (RRF), rerank cross-encoder fp16, model server HTTP, đánh giá có bootstrap CI và McNemar. Điềm sâu nhất có thể giải thích và sửa tại chỗ: chưa chốt |

Bảng B.1: Tự kiểm ba tầng.

## PHỤ LỤC C. THỰC NGHIỆM HỌC TẬP

Các thí nghiệm dưới đây chỉ dùng để học hoặc để ra quyết định, không khai trong Bảng phủ kỹ thuật.

- **Hybrid ưu tiên vector 3:1** (`relative_score`): tốt nhất trên bộ 01 ( $R@1$  84%) nhưng kém vector trên bộ 02; kết quả ở `eval/results/truy-hoi-hybrid-w.md`.
- **Hai mô hình embedding**: `profile local` (`vietnamese-bi-encoder`, 768 chiều, nhận tối đa 256 token, cần tách từ) cho Recall thấp hơn hẳn `profile server`. Số đo cũ ( $R@1$  64,5% trên 31 câu) thuộc pipeline một văn bản 21 Node nên không so trực tiếp được với số hiện tại. (*Chưa làm — đo lại trên corpus 54 Node ở tuần 6–7*)
- **Chất lượng OCR**: tỷ lệ âm tiết tiếng Việt hợp lệ dùng để chọn giữa các bản scan; QĐ 3183/2025 thấp nhất (94,9%).